

中图分类号 _____

UDC 分类号 _____

学校代码 _____ 10459 _____

密 级 密/内部/公开 _____



郑州大学
ZHENGZHOU UNIVERSITY

本科毕业设计

基于用户画像的弹幕视频网站设计与实现

学 生 姓 名：曾池

学 号：202234000101

导师姓名、职称：徐爱琴 副教授

学 科 专 业：管理学

研 究 方 向：信息管理与信息系统

培 养 单 位：信息管理学院

完 成 时 间：2026 年 05 月

A Dissertation/Thesis Submitted to
Zhengzhou University
for Bachelor Degree

Design and Implementation of a Danmaku Video
Website Based on User Profiling

By Chi Zeng

Supervisor: Prof. Aiqin Xu

Information Management and Information System
School of Information Management

May, 2026

郑州大学

学位论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全知晓本声明的法律后果由本人承担。

学位论文作者签名：曾池

指导教师签名：

日期： 年 月 日

郑州大学

学位论文使用授权书

本人同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权郑州大学可以将本学位论文的全部或部分编入有关数据库进行检索，可以采用影印、缩印或者其他复制手段保存论文和汇编本学位论文。

学位论文作者签名：曾池

指导教师签名：

日期： 年 月 日

中文摘要

随着弹幕视频网站的快速发展，弹幕交互已成为用户参与视频内容讨论的重要方式。然而，有特定信息需求的用户面对视频在短时间内产生的大量弹幕常出现信息过载问题，难以从中快速筛选出所需的信息。针对这一问题，本文设计并实现了一个基于用户画像的弹幕视频网站，通过构建用户画像模型优化弹幕推送机制。

本文采用前后端分离架构，前端基于 Vue3 框架构建，后端使用 Spring Cloud 微服务框架，用户画像引擎基于 Python 语言与 Flask 框架开发。系统从兴趣偏好、行为属性、弹幕质量三个维度构建用户标签体系，其中兴趣标签融合了阿里云百炼向量大模型的语义匹配与关键词匹配策略，行为标签基于统计分布与阈值判定，质量标签通过多维度加权评分计算。利用构建好的画像，系统提供标签筛选弹幕、查找相似用户等功能，提高了信息获取效率，并扩展弹幕社交应用场景。

测试结果表明，系统成功实现了弹幕系统与用户画像的结合应用，并能在稳定运行的基础上满足高并发等非功能性需求。本文工作为视频网站强化弹幕推送机制、提升用户信息获取体验提供了可行方案。

关键词：用户画像；弹幕；视频网站；Spring Boot

ABSTRACT

With the rapid development of danmaku video websites, danmaku interaction has become a key means for users to engage in video content discussions. However, users with specific information needs often face information overload due to the large volume of danmaku generated within a short time, making it difficult to quickly filter out the information they require. To address this issue, this paper designs and implements a danmaku video website based on user profiling, which optimizes the danmaku recommendation mechanism by constructing user profile models.

This paper adopts a front-end and back-end separation architecture. The front-end is built on the Vue3 framework, the back-end is developed using the Spring Cloud microservice framework, and the user profile engine is implemented with Python and the Flask framework. The system constructs a user tag system from three dimensions: interest preference, behavioral attributes, and danmaku quality. Specifically, interest tags integrate semantic matching from Alibaba Cloud Bailian's vector large model with keyword matching strategies; behavioral tags are derived from statistical distribution and threshold-based determination; quality tags are computed through multi-dimensional weighted scoring. Leveraging the constructed profiles, the system provides functions such as tag-based danmaku filtering and similar user discovery, thereby enhancing information retrieval efficiency and extending danmaku's social application scenarios.

Test results show that the system successfully integrates danmaku functionality with user profiling, and is capable of meeting non-functional requirements such as high concurrency while maintaining stable operation. This work offers a feasible solution for video websites to enhance danmaku recommendation mechanisms and improve user experience in information acquisition.

Key words: user profiles; danmaku; video website; Spring Boot

目 录

中 文 摘 要	I
ABSTRACT.....	II
目 录	III
第 1 章 绪论	1
1.1 研究背景	1
1.2 研究现状	2
1.3 研究内容	3
1.3.1 研究内容与研究方法	3
1.3.2 论文结构安排	4
第 2 章 相关技术概述	5
2.1 SPRING BOOT 框架	5
2.2 数据库与中间件	5
2.2.1 MySQL 数据库	5
2.2.2 Redis 数据库	5
2.2.3 Elasticsearch 搜索引擎	6
2.2.4 消息队列 RabbitMQ	6
2.3 用户画像	6
第 3 章 基于用户画像的弹幕视频网站需求分析	8
3.1 功能性需求分析	8
3.1.1 弹幕视频网站功能性需求分析	8
3.1.2 支持用户画像的弹幕模块功能性需求分析	9
3.2 非功能性需求分析	10
第 4 章 基于用户画像的弹幕视频网站设计与实现	12
4.1 系统架构设计	12

4.2 数据库设计	16
4.2.1 概念结构设计	16
4.2.2 物理结构设计	16
4.3 用户画像引擎设计与实现	23
4.3.1 用户画像引擎设计	23
4.3.2 用户画像引擎实现	29
4.4 后端业务服务的设计与实现	33
4.4.1 登录鉴权模块设计与实现	34
4.4.2 点赞高并发优化设计与实现	36
第 5 章 基于用户画像的弹幕视频网站测试	39
5.1 测试和开发环境概述	39
5.2 功能性测试	39
5.2.1 用户画像功能性测试	40
5.2.2 后端业务功能性测试	41
5.3 非功能性测试	42
第 6 章 总结与展望	46
参 考 文 献	47
致 谢	49

第1章 绪论

1.1 研究背景

随着互联网的迅速发展,近年来诞生了一批以 Bilibili、AcFun 等为代表的弹幕视频网站,并在国内掀起一波弹幕文化热潮。这类平台的弹幕交互系统普遍参与门槛低、支持实时渲染,因此热门视频往往能在短时间内收获数量巨大的弹幕信息,在营造亲切热烈的社交氛围的同时,也带来了信息过载的问题。有特定信息需求的普通用户难以在海量的弹幕中快速、精准地筛选出自己期望获取的信息。如何优化弹幕信息推送、满足用户多样化的信息获取需求,是目前弹幕视频网站亟待解决的问题。

用户画像自 2007 年由 Cooper A 等提出以来^[1],已在精准营销、个性化推荐、数据挖掘与分析等领域得到广泛应用。其核心理念与技术方法,显然也适用于优化弹幕信息推送这一具体场景。平台通过收集用户的弹幕发布记录、互动行为(如点赞、回复)以及弹幕所属视频的元数据(如标题、分区、标签)完成数据收集,接下来需要进行数据挖掘,结合发布弹幕的用户的过往行为提取兴趣维度标签、信息行为标签、弹幕质量标签,进而构建用户画像模型,并将结果反馈给用户,让用户快速获悉弹幕话题构成与发布弹幕的用户群体结构。

目前市面上知名的主推或支持弹幕交互系统的视频网站普遍缺乏针对弹幕数据的推送优化,比如 Bilibili 和 AcFun,只是将弹幕简单罗列在列表里呈现给用户。因此,本文将在构建基于 Spring 框架实现的视频平台的基础上,实现用户画像与弹幕交互系统的初步结合应用,该应用蕴含以下几大潜在优势:

(1) 满足不同层次用户多样化的信息获取需求。对普通观众而言,他们可以借助画像标签筛选出自己感兴趣的弹幕,甚至找到兴趣相投的同好,从而在观看视频的同时满足一定的社交需求;针对创作者,可以从中获得过程性创作反馈,这将有别于传统的只关注播放量、收藏量等的结果性反馈。

(2) 提升平台竞争力,改善平台生态。以 Bilibili 为例,其过往的视频互动主要聚焦于评论区,弹幕列表默认隐藏,用户容易失去对弹幕信息的深度关注;B 站实际上在后台对创作者开放详细的数据分析,非创作者难以主动便捷地进行数据分析。本文结合前端可视化技术,有望提升平台“数据智能”竞争力,让数据对广大用户更为“触手可及”,推动数据的“普惠化”与“易用化”。

(3) 广阔的市场前景。Bilibili 在 2025 年上半年的财报中指出, 仅平台日活用户就达到 1.088 亿, 同比增长 6%, 每位活跃用户日均使用时长达 106 分钟, 这显示其作为行业代表性企业具有巨大的影响力和高度用户粘性, 在此背景下, 推广弹幕推送优化显然有广阔的市场前景^{[2][2]}。

1.2 研究现状

世界上最早的在线流媒体视频可以追溯到 1995 年 Progressive Networks 公司发布的比赛实况转播, Progressive Networks 公司就是后来掀起在线流媒体视频潮流的 RealNetworks^{[3][4]}。经过多年的发展, 如今主流的在线视频平台按内容生产模式可被分为三大类: UGC (User-generated Content)、PGC (Professionally Generated Content) 和 OGC (Occupationally Generated Content)。其中 UGC 意指用户内容生产, 允许用户上传、分享原创或二次创作内容, 涵盖视频、直播等多种形式; PGC 主要以电视剧、电影等视频输出模式为主; OGC 的生产主体是从事相关领域工作的专业人员^{[5][6]}。由于本文构建的视频网站以 UGC 模式为主, 并且如今弹幕文化较为发达的视频网站主要集中在 UGC 模式, 因此下文将着重探讨这类视频网站。

国内外公认的首个在技术上实现 UGC 模式的视频网站为 2005 年上线的 ShareYourWorld.com, 而首个在该模式下取得巨大成功的视频网站则是 2005 年 4 月 23 日上线的 YouTube。我国创业者同样紧跟这一趋势。2005 年, 王微创立土豆网, 随后优酷网、酷 6 网等视频网站相继上线。

2006 年 Niconico 动画网站成立, 其被广泛认为是弹幕视频网站的始祖, 同时也是“弹幕”一词首次出现的地方^[7]。Niconico 同样采用了 UGC 模式, 允许用户上传自制视频。国内最早引入类似的弹幕系统的视频网站是 2007 年成立的 AcFun, 后来 AcFun 在商业化、产品运营和内容生态拓展上逐渐落后于 2009 年成立的 Bilibili。Bilibili 继承了 AcFun 部分商业模式, 将弹幕文化在国内发扬光大, 逐渐成为弹幕视频网站的领军者。这类网站不仅在技术上引入了弹幕系统, 还在多年的发展过程中衍生出富有活力的弹幕文化。

目前设计并实现 UGC 视频网站, 同时结合用户画像对网站中的弹幕系统进行改造的相关研究较少。一个在工程实践上较为相似的研究是钟潇萌基于 Spring Boot 实现了一个具备弹幕互动功能的视频网站, 并着重优化了个性推荐模块^[8], 但该模块没有引入用户画像, 且其旨在向用户推荐视频, 而非帮助用户筛选弹幕。其余研究更多是基于现存网站的弹幕数据构建用户画像, 研究目的也不一定是为了优化弹幕推送。比如姚苏梅基于爬取的弹幕信息, 提出 RMS 用户画像模型构建方法, 为个性化服务与精准推荐提供支持^[9]; 郭

强基于弹幕构建“双碳”科普视频用户画像，旨在为政策制定者推进“双碳”目标提供实证参考与建议^[10]。

关于用户画像，自 Cooper A 等在 2007 年提出这一概念以来^[1]，其已在旅游、金融理财、营销推荐等许多领域取得了精彩的应用成果，比如袁航等针对电商场景中用户画像构建时传统 TF-IDF 忽略标签相关性及场景特性的问题，提出改进的用户标签权重计算方法，有效提高了推荐准确性^[11]。一些研究针对在线文本数据进行用户画像模型的构建，比如 Sloan L 等通过将 Twitter 用户的个人简介与国家标准职业分类进行匹配，实现了对用户年龄、职业与社会阶层的自动化推断，为从短文本中提取结构化用户标签提供了借鉴^[12]。然而，直接从弹幕文本这一富含用户情感与兴趣的短文本数据中构建用户画像的研究，目前仍较为匮乏。

综上所述，目前对视频网站的弹幕推送机制优化的研究较为匮乏，因此本文创新性地应用弹幕信息对用户进行刻画，具有应用价值。本文主要根据用户过往发布过的弹幕，从兴趣爱好、信息行为、弹幕质量三个维度构建用户标签，形成用户画像模型。此时，前端网页就可以根据标签对弹幕进行筛选，例如只显示拥有“高质量”标签的用户的弹幕。

1.3 研究内容

1.3.1 研究内容与研究方法

本文基于广泛使用的 MVC 模式设计了一个弹幕视频网站，并利用 Vue3、Spring Boot 和 Mybatis-Plus 等技术实现了该系统。随后经过文献查阅与亲历测评，发现主流弹幕视频网站的弹幕系统存在不足：在观看视频的过程中，有特定信息需求的用户面对短时间内涌现的大量弹幕可能难以快速筛选，出现信息过载问题。目前针对该领域进行优化的研究较为匮乏，于是本文提出一种结合用户画像的弹幕推送优化方案，并借助 Python 语言、Elasticsearch 搜索引擎、云平台部署的向量大模型等技术实现该用户画像引擎。

本研究预计产出以下形式成果：

（1）搭建弹幕视频网站系统，同时提供源代码与相关资源文件，涵盖前端网页、后端业务服务和用户画像引擎三个子系统。

（2）毕业设计论文一篇。论文将系统阐述研究背景、目标、关键技术、系统设计与实现过程、测试结果等，并进行总结与展望。

1.3.2 论文结构安排

本文结构安排如下：

第一章为绪论，介绍了论文的研究背景，即在弹幕视频网站迅速发展的当下，结合用户画像改善其弹幕系统存在的用户信息过载问题。接着梳理了弹幕视频网站的发展脉络，并简短介绍了与弹幕相关的用户画像方面的研究。此外还讲解了论文的研究内容、研究方法 with 结构安排。

第二章为相关技术概述，着重介绍了在构建弹幕视频网站的过程中使用到的主要技术及采用此技术的理由。

第三章将概述网站业务、分析网站需求，其中需求分析的重点在于站在用户角度讨论网站需求来源与优化弹幕系统的必要性。

第四章讲解网站设计与实现的具体过程。其中包括系统架构设计、功能划分、主要业务流程和数据库设计等多方面内容。

第五章主要展示网站开发环境、网站实现效果与测试结果。

第六章为总结与展望，将回顾论文内容与研究过程，对全过程做一个总结，并展望网站可能的改进方向。

第2章 相关技术概述

2.1 Spring Boot 框架

为解决传统 Spring 项目在创建时需要编写大量配置文件的问题，Spring Boot 应运而生^[13]。它简化了复杂的配置文件，内嵌 Web 容器，打包和部署十分方便，同时拥有完善庞大的技术生态，这些优点使得它迅速成为 Java 领域主流的后端开发框架。

为了提高网站系统的扩展性和可维护性，本文采用微服务架构搭建系统。微服务的概念于 2012 年被 Martin Fowler 提出，其将系统根据业务需求划分为多个服务，每个服务仅负责其对应的业务，每个服务都可以单独运行并独立部署^[15]。由于不同服务松耦合，支持多节点同时部署，所以这种架构具备高可靠性、并发量大的特点，后期扩展新功能也方便。同样的，为了减轻开发者的负担，在 Spring Boot 的基础上诞生了 Spring Cloud 开发脚手架，其封装了服务发现、负载均衡、熔断等一系列实现微服务的关键操作^[16]，可以快速构建一个微服务系统，这也是本文采用 Spring Cloud 构建网站后端微服务系统的重要原因。

2.2 数据库与中间件

2.2.1 MySQL 数据库

MySQL 是关系型数据库管理系统，在众多数据库中，其因开源免费、使用简便、功能强大等多种优点备受青睐^[17]。MySQL 基于 B+树构建索引，实现高效的数据存储与检索。在对数据进行复杂读写操作时，MySQL 使用事务管理 SQL 语句的执行，而且为了实现高并发事务控制，MySQL 利用事务隔离机制来保证事务原子性、数据一致性和修改持久性，而隔离机制底层又是通过复杂的锁机制和 MVCC 机制实现的。本文构建的网站系统采用 MVC 思想设计，MVC 即模型 Model、视图 View、控制器 Controller 的缩写，其中模型 Model 需要采用面向对象方法，将问题中的对象抽象成程序对象，并封装其属性和逻辑^[18]，适合采用关系型数据库存储。系统的部分业务场景针对存储的数据有复杂的并发操作，因此采用 MySQL 数据库是经济、合理的选择。

2.2.2 Redis 数据库

Redis 数据库是开源免费的键值对存储系统，它主要将数据存储在内存中，

可实现每秒超 100K 的吞吐量，还具备较完善的数据持久化机制，可通过集群部署、主从同步保证数据安全性。在本文中，Redis 主要承担以下工作^[19]：

- (1) 利用其存取速度快的特点，实现部分业务的高并发数据存取。
- (2) 存储可能需要周期刷新或者生命周期较短的热点数据，比如热搜列表。
- (3) 作为不同服务间信息交互的桥梁，例如实现后端微服务结点和用户画像服务之间的通信。

2.2.3 Elasticsearch 搜索引擎

Elasticsearch 是基于 Lucene 构建的高可扩展、高实时性搜索服务器，是开源的大数据搜索引擎解决方案中最受欢迎的之一^[19]。本文在构建用户画像时需要将部分数据以向量化形式存储起来，而 Elasticsearch 支持向量检索与存储，且有 HNSW 算法等机制保证高效检索，因此被本文引入。

2.2.4 消息队列 RabbitMQ

RabbitMQ 是基于高级消息队列协议标准设计的消息队列系统，同样开源免费。RabbitMQ 可帮助开发者轻松实现异步消息传递、跨应用通信和业务解耦等功能^[19]。例如，用户端发起构建用户画像的请求后，需要等待服务端进行漫长、复杂的运算才能得到返回结果，响应时间过长，有损用户体验。引入 RabbitMQ 后，用户端只需把请求放入消息队列排队，就可以直接返回，无需关心后续冗长的计算过程。消息队列会自动选择合适的时机和健康的服务结点把这个构建用户画像的请求“传达”出去，服务端收到“通知”后会构建用户画像，并把结果存入指定的数据库等待用户端读取，实现了异步处理、降低了服务器响应时间。当短时间内发送的请求过多时，消息队列还起到数据缓冲的作用，避免服务结点或数据库因承受不住大量请求出现错误。此外，RabbitMQ 消息队列拥有较完善的消息可靠性保证机制，支持插件扩展，可以很好地帮助实现本文内部的业务功能。

2.3 用户画像

用户画像是在收集、清洗和分析用户数据的基础上，从中提取用户标签，并利用这些标签描述用户的技术。通常构建用户画像分为以下几大步骤：

- (1) 数据收集。通过爬虫、记录日志等手段收集与用户相关的数据，将其存储在数据库中为接下来的步骤做准备。在本文中，用户发送的弹幕、视频观看记录、视频交互记录（包括点赞、投币、收藏等）会被自动录入 MySQL

数据库，它们与视频信息结合构成本文主要的数据来源。

（2）构建标签体系。用户画像本质上是将用户数据以标签化形式展示，构建完备的标签体系有助于全面、立体地描绘用户特征。数据来源不同，标签的构建方法也会有差异。年龄、性别、国家等不轻易发生改变的静态数据本身可直接作为标签使用。而动态数据主要产自用户行为，可以通过统计、设计规则和使用特定算法等方法从中提取标签^{[11][20]-[21]}。

（3）构建用户画像。根据设立好的标签体系，将预先收集到的数据经过清洗、转换、分析和挖掘等处理后，可以得到用户标签，这些标签已经足够描绘用户画像。如果有需要，还可以使用归一化、权重计算、聚类等手段进一步处理标签，为特定的场景服务。

（4）画像应用。用户画像常用于精准营销、决策辅助、算法优化等领域，在本文中则用于优化弹幕推送。

在构建用户画像的过程中，本文拟用阿里云百炼平台的 `text-embedding-v4` 向量大模型进行用户数据与标签的文本向量化与语义匹配。阿里官方并未公布该大模型的技术细节，但从社区反馈和开发测试过程的使用体验来看，该模型能够结合上下文精准理解语义，且 API 调用响应速度较快，能胜任企业级 RAG 项目开发，适合引入本文的用户画像模块。

第3章 基于用户画像的弹幕视频网站需求分析

3.1 功能性需求分析

3.1.1 弹幕视频网站功能性需求分析

随着互联网的迅速发展，仅通过文字和图片等媒介已不能满足大众的网络社交需求，人们开始通过视频来记录生活、表达自我。视频媒介可为观看者带来远超其他媒介的丰富视听体验，并兼具更强的传播效果。本文搭建的弹幕视频网站便为用户提供了通过视频媒介触摸世界的宽广平台，同时由于其以 UGC 模式为主，面向全年龄段群体，允许用户基于自身兴趣爱好上传自制视频，可以满足不同的喜好，并激发更多人尝试视频创作，形成“创作——分享——再创作”的良性内容生产循环。此外，网站搭载评论互动模块和弹幕交互模块，能结合视频内容创造出沉浸式的社交氛围，满足用户的社交需求。拥有类似内容创作模式的 Bilibili 和 YouTube 均取得商业上的巨大成功，月活流量和内容丰富度屡屡创新高，证明了弹幕视频网站拥有旺盛的需求和广阔的市场前景。

网站用例图如下：

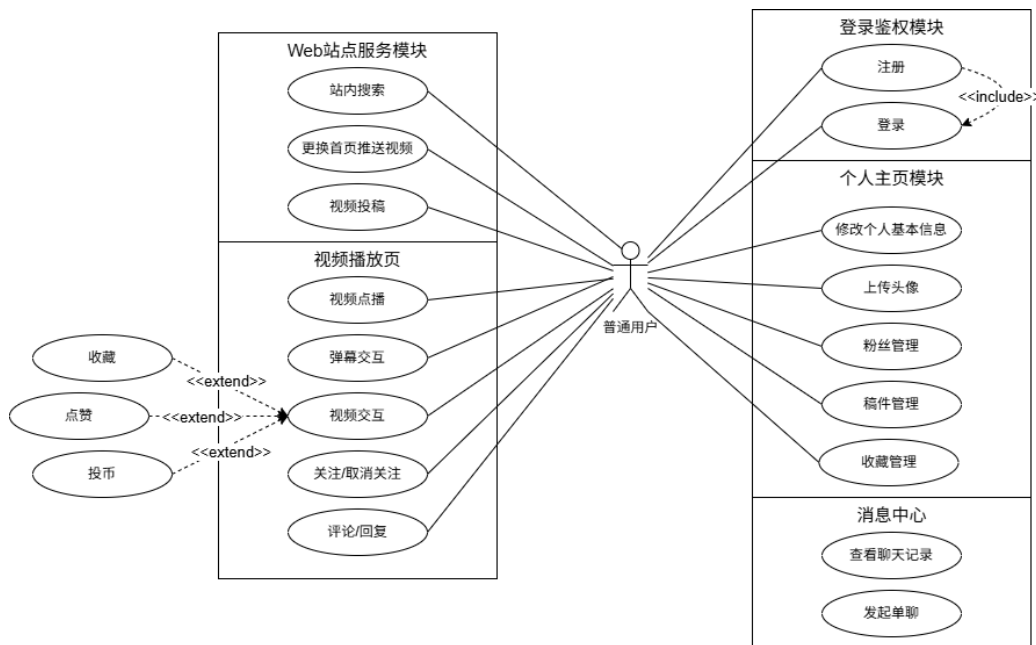


图 3.1 弹幕视频网站 UML 用例图

网站主要包含以下几大模块：

- (1) Web 站点服务模块，包括站内搜索、更换首页推送视频、视频投稿

等功能，其中站内搜索可根据关键字搜索全站视频稿件，支持按照发布时间、播放量、收藏量等字段排序搜索结果。

(2) 登录鉴权模块，包括注册和登录两个独立功能。其中注册功能若成功会自动登录。

(3) 视频播放页，包括视频点播、弹幕交互、视频交互、关注/取消关注（针对视频作者）、评论/回复等功能。其中视频交互包括点赞、投币、收藏三大功能。

(4) 个人主页模块。用户可在此修改个人基本信息，比如个性签名、昵称、公告栏等，还可以修改头像；可以在个人主页查看关注自己的人或自己关注的人；可以查看自己投稿过的视频，支持按收藏量、播放量等字段对搜索结果排序；可以管理收藏夹，并对收藏夹和收藏夹内部的视频进行增删改查等操作。

(5) 消息中心。用户可在此查看与某位用户的聊天记录，或者发起新聊天。

3.1.2 支持用户画像的弹幕模块功能性需求分析

现存的主流弹幕视频网站的弹幕系统容易使拥有特定信息需求的用户产生信息过载问题，其展示弹幕的方式主要有两种：实时悬浮在视频画面中和集中展示在右侧列表。悬浮方式会将在当前视频播放时间节点附近发布的所有弹幕一并加载出来，若弹幕数量过多，弹幕会互相遮盖，用户难以在满屏重复、交叉重叠的冗余弹幕之间提取自己需要的信息。如果弹幕是滚动式弹幕，在视频画面的停留时间较短，用户还需要反复暂停视频停止弹幕滚动。在学习类视频，这类用户信息过载的问题会更为凸显。右侧列表方式会将本视频某个时间段内的弹幕集中列出，若用户有特定信息需求，依旧需要在大量重复冗余的弹幕间做筛选。

主流弹幕视频网站普遍引入弹幕屏蔽机制，具体形式有按照关键词屏蔽、按照用户 ID 屏蔽、按照正则屏蔽等。以 Bilibili 为例，被屏蔽的弹幕不会出现在视频画面上，但会在右侧列表保留，并加上被屏蔽标识。该机制存在不够灵活的问题，若用户设置的规则不当，依旧会保留大量重复或低质量弹幕。

本文拟结合用户画像技术解决信息过载问题，可按照用户要求将具备特定标签的用户发布的弹幕优先推送，不再需要在视频画面或冗长的弹幕列表进行筛选。例如，系统事先对用户历史发送的弹幕进行质量评估，并根据评估结果提取用户标签。在网页前端，观看者可点选优先显示拥有“高质量弹幕提供者”标签的弹幕，此时右侧列表只会显示筛选后的结果，平均弹幕质

量较低的用户弹幕将被排除，做到智能集中推优。

此外，引入用户画像技术可以扩展弹幕信息的利用价值。例如，系统可以筛选出与观看者拥有相似爱好或共同关注点的用户的弹幕，帮助用户在海量弹幕信息中发现同好与相似观点，满足用户的社交需求，营造更具备沉浸感的互动氛围。

本文中弹幕模块的 UML 用例图如下：

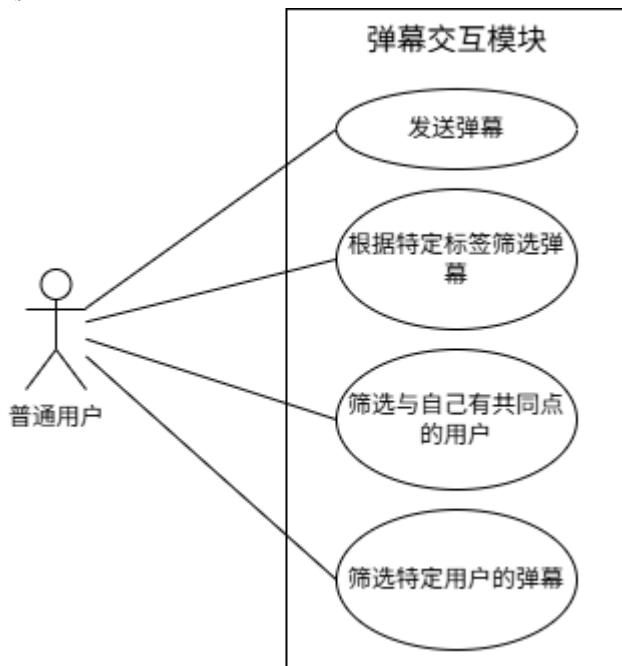


图 3.2 弹幕交互模块 UML 用例图

用户在该模块下可执行的功能：发送弹幕；根据特定标签查看被筛选后的弹幕；在本视频发送过弹幕的所有用户间，查找与自己标签相似的用户，并进一步查看这些用户的弹幕。

3.2 非功能性需求分析

一套完整的系统除了实现基本功能，还需确保能在多变、复杂的生产环境持续良好运行，主要包含以下几大方面：

（1）高可用。网站服务需要确保长时间不宕机、不重启、不故障，否则轻则降低用户体验，重则丢失数据，造成数据安全问题 and 经济损失。在数据层面，需要视具体情况保证数据一致性和持久性，例如在对一致性要求较高的业务场景，需确保数据在 Redis 和 MySQL 两类数据库的读写一致；确保 RabbitMQ 消息可持久化存储，以便消息丢失或者服务宕机重启能及时恢复数据。

（2）高并发。随着用户量的增长，网站服务在单位时间内处理的网络请

求必然水涨船高，服务需要在这种情况下保证稳定运行和低响应时间，提高用户体验。

（3）可扩展。随着时间推移和市场环境变化，弹幕视频网站可能面临需要扩展业务的场景，此时系统最好保留扩展性，便于增加新功能以节省开发成本，而非推倒重做。还要注意新增功能不能对现存功能造成过大干扰，影响服务运行。

（4）数据安全。网站存在登录鉴权业务，这是隐私泄露的高发场景，需要确保用户在登录注册时的隐私安全。此外其他场景的数据安全也不容小觑，比如防范恶意网络攻击、保障数据库信息不外泄、防范系统内部越权违规操作等。

第4章 基于用户画像的弹幕视频网站设计与实现

4.1 系统架构设计

在逻辑层面，软件工程领域常常采用分层架构剖析系统设计，比如 Fowler M 在《企业应用架构模式》中提出企业应用可划分为表现层（Presentation Layer）、领域层（Domain Layer）和数据源层（Data Source Layer）三层^[22]。分层设计系统架构可将庞大的系统拆解为人脑易于理解的逻辑模型，每一层各司其职、减少耦合，提升了可维护性和扩展性。因此，本文参考该思想，将整体逻辑架构设计成如下图所示的形式：

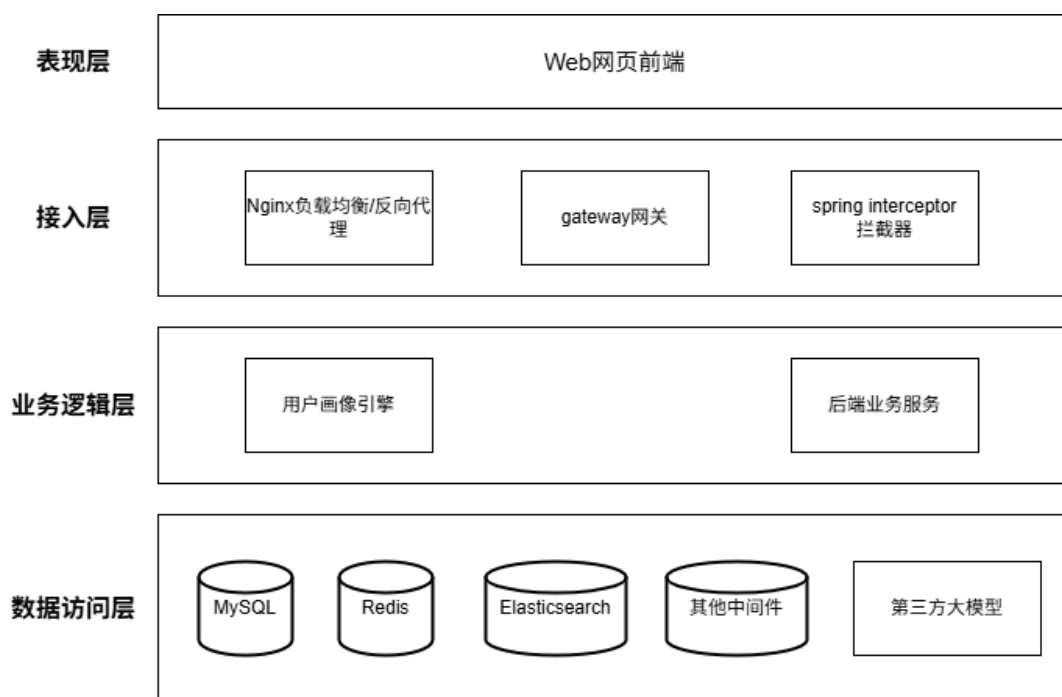


图 4.1 系统整体架构

表现层为基于浏览器的用户端，提供可视化界面方便用户与服务器交互，满足用户需求。

网络请求到达接入层后，依次经过 Nginx 服务、Gateway 网关和拦截器（interceptor）。其中 Nginx 聚焦基础访问控制和内外网隔离，Gateway 网关负责全局请求信息控制、全局日志记录和面向微服务的服务分发等，interceptor 拦截器负责细粒度的登录鉴权和用户信息解析。

业务逻辑层负责具体业务流程实现，是整个系统的核心。

数据访问层负责数据存取，其中由 Mybatis-Plus 框架负责业务逻辑层与 MySQL 数据库的对接。Mybatis-Plus 基于 Mybatis 进行二次封装而来，拥有

丰富灵活的对上数据存取接口和较完善的预编译 SQL 机制，在企业开发过程中被广泛采用。

接下来是用户画像引擎的总体架构设计和后端业务服务的总体架构设计，如图所示：

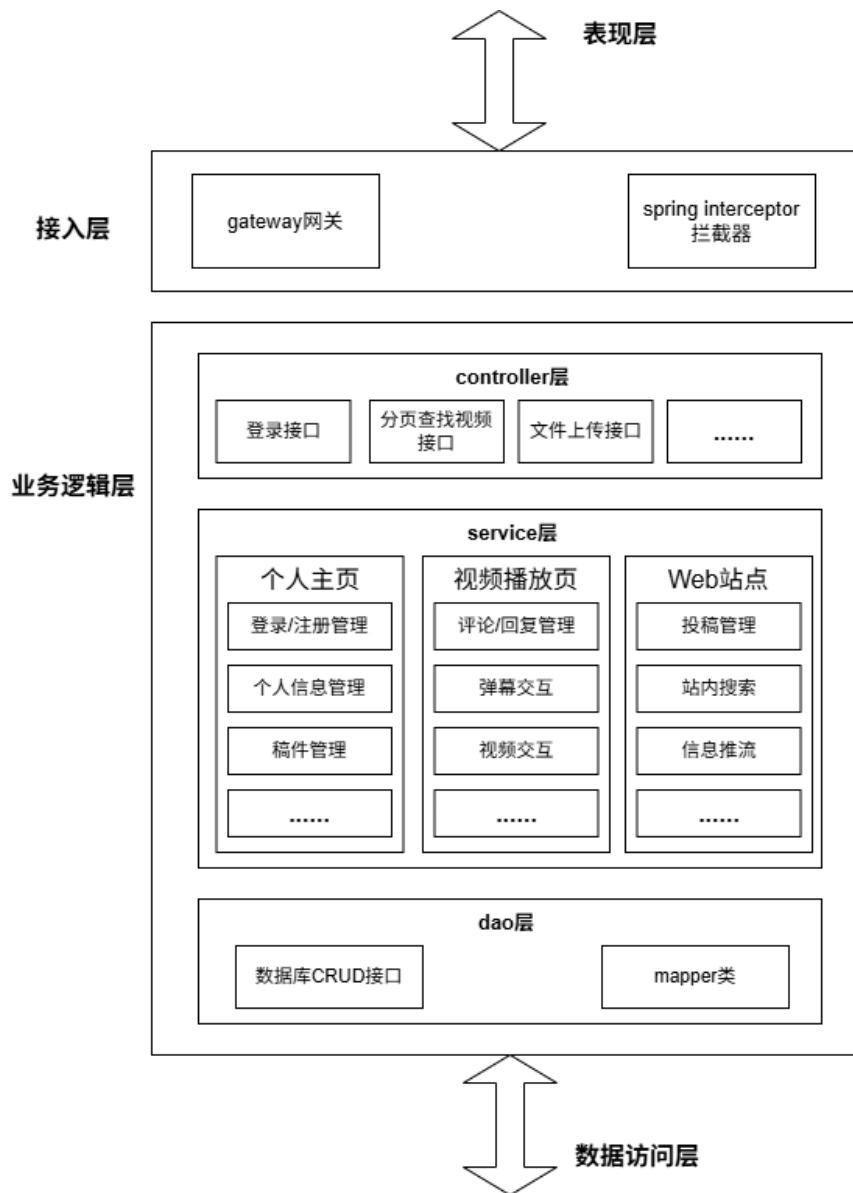


图 4.2 后端业务服务整体架构

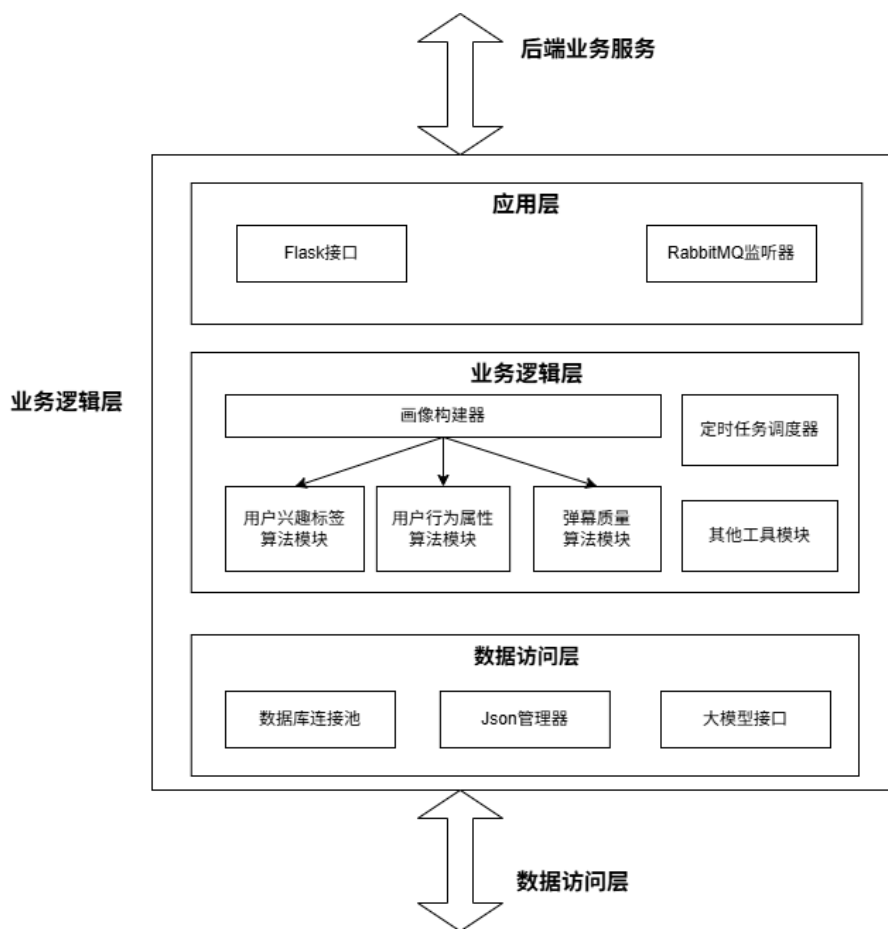


图 4.3 用户画像引擎整体架构

逻辑层面的分层设计和模块划分实际上是为了方便人脑理解，在物理层面，本文搭建的完整系统包括三个子系统：前端网页、后端业务服务和用户画像引擎，以及数据库和各类中间件。它们需要部署在不同的网络节点，组成去中心化的简单分布式系统。不同系统通过发送网络请求或者访问中间件服务进行通信。其中后端业务服务借助 Spring Cloud 构成微服务，本身也是分布式系统，其每个子模块可单独部署在一个网络节点，不同模块之间尽可能降低耦合度、提高内部聚合度。整体架构如下图：

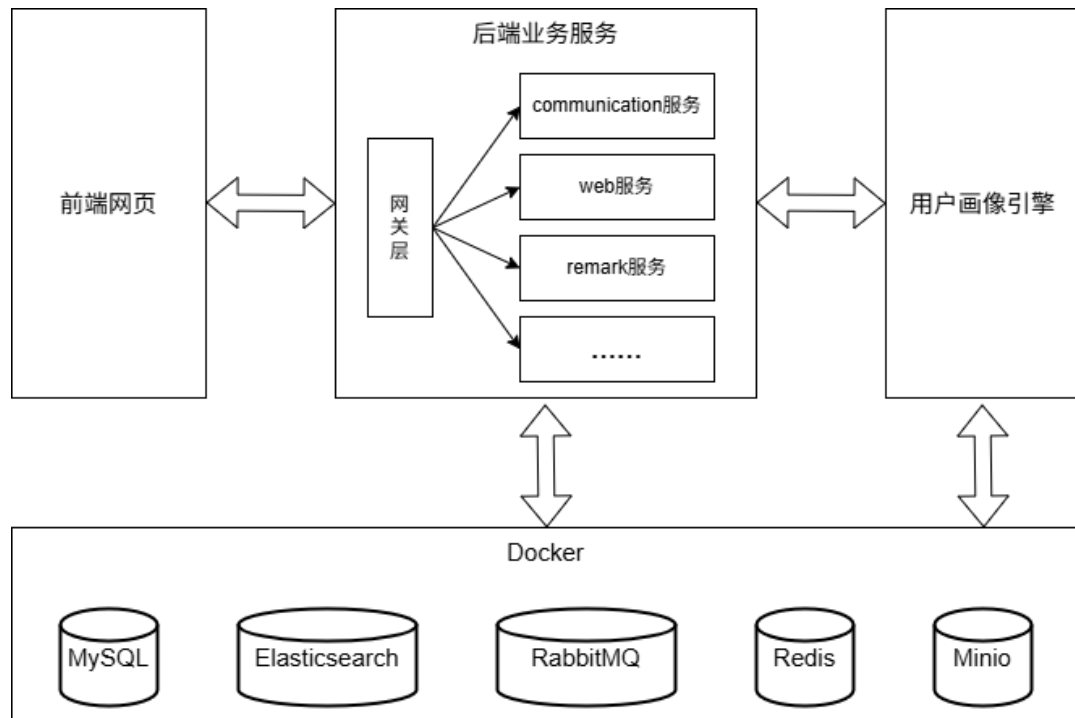


图 4.4 系统物理架构设计

系统的后端模块设计如下图：

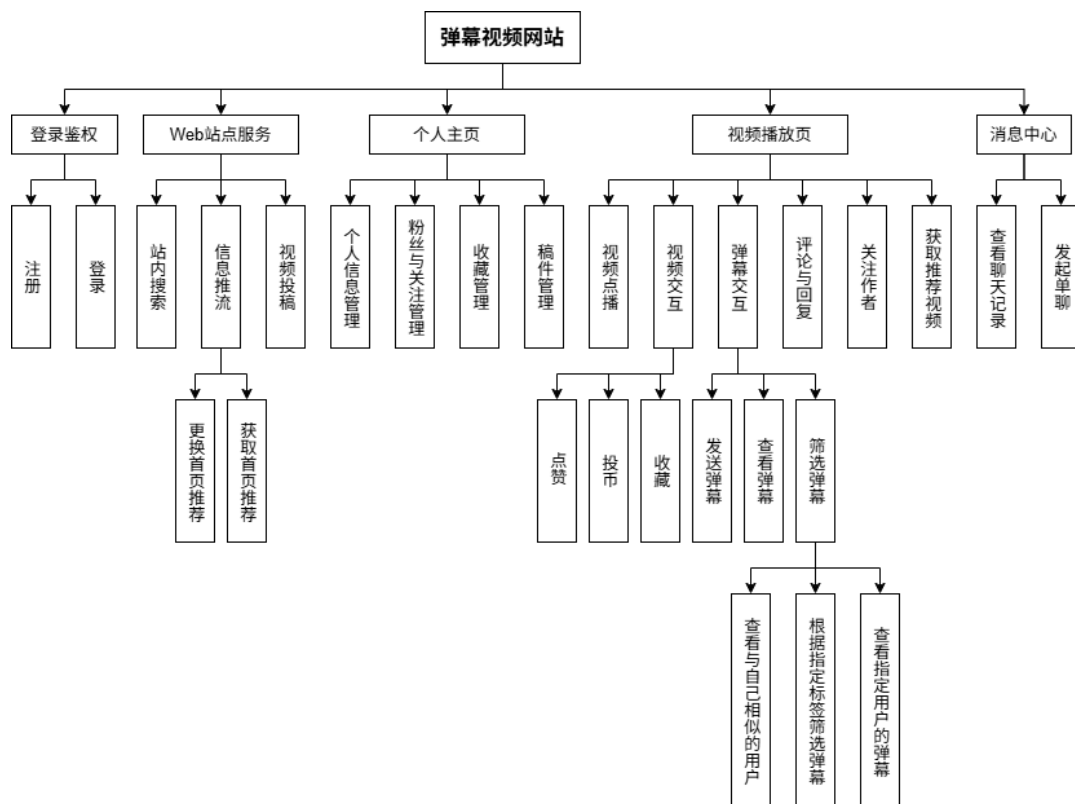


图 4.5 模块设计

4.2 数据库设计

这里主要介绍关系型数据库 MySQL 的设计。

4.2.1 概念结构设计

本文的实体的概念结构设计如下图：

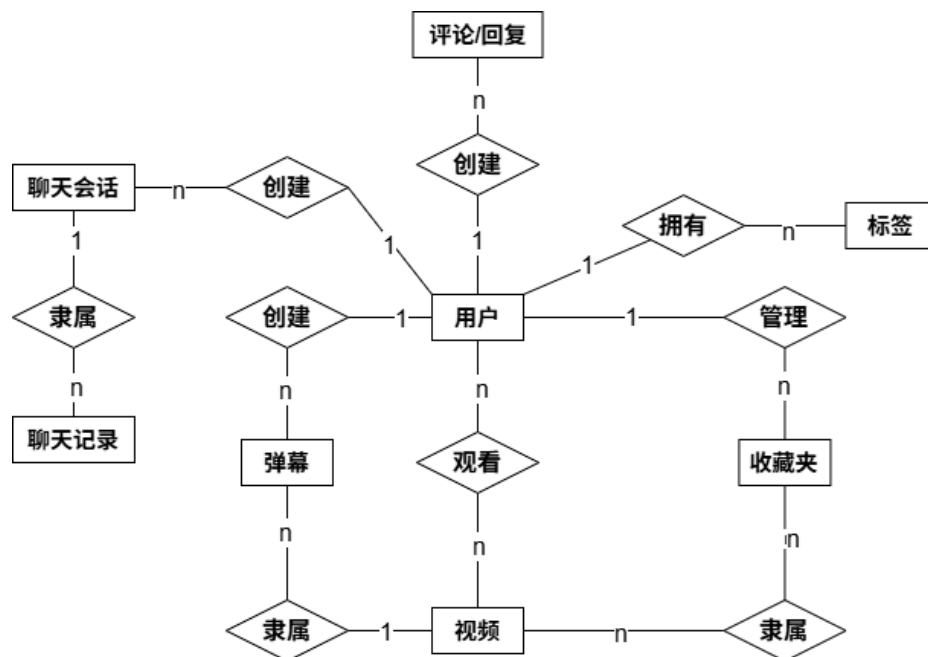


图 4.6 简化的数据库 E-R 图

由于部分实体的属性数量较多，单个实体可能有近 30 个属性，为了保证图片美观与清晰，在 E-R 图中暂时隐去实体属性。每个实体的属性将在 4.2.2 物理结构设计中详细展示。

4.2.2 物理结构设计

数据库物理结构设计如下：

（1）用户视频浏览历史（bh_video）：

表 4.1 用户视频浏览历史

字段名	字段解释	数据类型	允许空值	备注
id	ID 值	bigint unsigned	N	主键，自增
uid	用户 ID	bigint unsigned	N	
vid	视频 ID	bigint unsigned	N	
view_time	浏览时间	datetime	N	
duration	上次观看到的 时间节点	double	N	

续表 4.1 用户视频浏览历史

字段名	字段解释	数据类型	允许空值	备注
is_finish	是否看完	tinyint	N	0-未看完, 1-已看完
device	设备信息	varchar(64)	Y	
ip	IPv4 地址	varchar(35)	Y	
is_delete	是否删除	tinyint	N	0-未删除, 1-已删除

(2) 聊天会话表 (chat), 表示两个用户间存在一对一聊天关系:

表 4.2 聊天会话表

字段名	字段解释	数据类型	允许空值	备注
id	ID 值	bigint unsigned	N	主键, 自增
target_id	对象 UID	bigint unsigned	N	
sender_id	发起者 UID	bigint unsigned	N	
latest_time	该会话最新的聊天记录的发送时间	datetime	N	

(3) 聊天记录表 (chat_message), 记录会话内部具体的聊天记录:

表 4.3 聊天记录表

字段名	字段解释	数据类型	允许空值	备注
id	ID 值	bigint	N	主键, 自增
sender_id	发起者 UID	bigint unsigned	N	
target_id	接收者 UID	bigint unsigned	N	
content	消息内容	varchar(500)	N	
sender_del	发送者是否删除	tinyint	N	0-未删除, 1-已删除
target_del	接收者是否删除	tinyint	N	0-未删除, 1-已删除
withdraw	发送者是否撤回	tinyint	N	0-未撤回, 1-已撤回
time	消息发送时间	datetime	N	
message_type	消息类型	tinyint	N	0-文本; 1-图片; 2-文件
read_status	是否已读	tinyint	N	0-未读; 1-已读

(4) 评论/回复表 (comment), 记录评论区的评论或回复:

表 4.4 评论/回复表

字段名	字段解释	数据类型	允许空值	备注
id	ID 值	bigint	N	主键, 自增
vid	视频 ID	bigint unsigned	N	
uid	发送者 ID	bigint unsigned	N	
root_id	根评论的 ID	bigint	N	
parent_id	父评论 ID	bigint	N	只有 root_id 为 0 时才允许为 0, 此时表示其为根评论
to_user_id	被回复用户 ID	bigint unsigned	N	
content	评论内容	varchar(2000)	N	
love	点赞数	int	N	
bad	点踩数	int	N	
create_time	创建时间	datetime	N	
is_top	是否置顶	tinyint	N	0-是; 1-否
is_deleted	是否删除	tinyint	N	0-是; 1-否

(5) 弹幕表 (danmu):

表 4.5 弹幕表

字段名	字段解释	数据类型	允许空值	备注
id	ID 值	bigint	N	主键, 自增
vid	视频 ID	bigint unsigned	N	
uid	发送者 ID	bigint unsigned	N	
fontsize	字体大小	tinyint	N	
mode	弹幕模式	tinyint	N	0-滚动; 1-顶部; 2-底部
color	弹幕颜色	varchar(7)	N	6 位十六进制标准格式
content	弹幕内容	varchar(100)	N	
time_point	弹幕出现时间 点	double	N	
status	弹幕状态	tinyint	N	1-通过审核; 2-审核中; 3-已删除
create_time	创建时间	datetime	N	

(6) 收藏夹表 (favorite):

表 4.6 收藏夹表

字段名	字段解释	数据类型	允许空值	备注
fid	ID 值	bigint	N	主键, 自增
uid	发送者 ID	bigint unsigned	N	
type	收藏夹类型	tinyint	N	1 默认; 2-用户自建
visible	是否公开	tinyint	N	0-隐藏; 1-公开
cover	收藏夹封面	varchar(255)	Y	
title	标题	varchar(20)	N	
description	简介	varchar(200)	Y	
count	内含视频数	int	N	
is_delete	是否删除	tinyint	N	0-是; 1-否

(7) 收藏夹与视频关联表 (favorite_video):

表 4.7 收藏夹与视频关联表

字段名	字段解释	数据类型	允许空值	备注
id	ID 值	int	N	主键, 自增
vid	视频 ID	bigint	N	
fid	收藏夹 ID	int	N	
time	收藏时间	datetime	N	
is_remove	是否删除	tinyint	N	0-是; 1-否

(8) 关注与被关注关联表 (follow):

表 4.8 关注与被关注关联表

字段名	字段解释	数据类型	允许空值	备注
id	ID 值	int	N	主键, 自增
follower_id	关注者 ID	bigint unsigned	N	
followee_id	被关注者 ID	bigint unsigned	N	
created_at	创建时间	datetime	Y	

(9) 用户表 (user), 存储与用户实体相关的较常用的基本信息:

表 4.9 用户表

字段名	字段解释	数据类型	允许空值	备注
uid	ID 值	bigint unsigned	N	主键
username	用户账号	varchar(50)	N	

续表 4.9 用户表

字段名	字段解释	数据类型	允许空值	备注
password	用户密码	varchar(255)	N	
nickname	用户昵称	varchar(32)	N	
avatar	头像 url	varchar(500)	Y	
background	主页背景图 url	varchar(500)	Y	
gender	性别	tinyint	N	0-男; 1-女
description	个性签名	varchar(100)	Y	
exp	经验值	int	N	
coin	硬币数	double	N	
				0-普通用户; 1-月度大会员; 2-季度大会员; 3-年度大会员
vip	会员类型	tinyint	N	
				0-正常; 1-封禁; 2-注销
status	账户状态	tinyint	N	
				0-普通用户; 1-管理员; 2-超级管理员
role	账户角色	tinyint	N	
				0-普通用户; 1-个人认证; 2-机构认证
auth	官方认证	tinyint	N	
auth_msg	认证说明	varchar(30)	Y	
create_date	账户创建时间	datetime	N	
delete_date	注销时间	datetime	Y	

(10) 用户信息表 (user_stats)。

该表同样存储与用户实体相关的基本信息, 和用户表 (user) 的区别在于该表主要存储查询频率相对较低的信息, 由于大部分查询集中在用户表 (user), 所以针对该表的查询大部分情况下采用懒加载机制。该设计将原本拥有 20 多个属性的大表拆分成两张小表, 降低了查询用户信息时的网络数据传输开销和磁盘 I/O 开销, 实现了冷热数据分离。该表物理结构设计如下表所示:

表 4.10 用户信息表

字段名	字段解释	数据类型	允许空值	备注
uid	ID 值	bigint unsigned	N	主键
masterpiece	置顶视频 ID	bigint unsigned	Y	

续表 4.10 用户信息表

字段名	字段解释	数据类型	允许空值	备注
fans_count	粉丝数	bigint	Y	
follow_count	关注数	int	Y	
like_count	获赞数	bigint	Y	
play_count	播放数	bigint	Y	
video_count	投稿数	int	Y	
collection_count	合集数	int	Y	
collect_count	收藏数	int	Y	
birth_date	出生日期	date	Y	
school	学校信息	varchar(64)	Y	
notice	公告	varchar(500)	Y	

(11) 用户标签表 (user_tag):

表 4.11 用户标签表

字段名	字段解释	数据类型	允许空值	备注
id	ID 值	bigint	N	主键, 自增
uid	用户 ID	bigint unsigned	N	
tag_name	语义标签名	varchar(50)	N	
weight	标签权重	float	N	
version	版本号	int	Y	方便日后适配 乐观锁
created_date	首次创建时间	timestamp	Y	
updated_date	最后更新时间	timestamp	Y	

(12) 用户与视频关联表 (user_video):

表 4.12 用户与视频关联表

字段名	字段解释	数据类型	允许空值	备注
id	ID 值	bigint	N	主键, 自增
uid	用户 ID	bigint unsigned	N	
vid	视频 ID	bigint unsigned	N	
play	播放次数	int	N	
love	是否点赞	tinyint	N	0-是; 1-否
unlove	是否点踩	tinyint	N	0-是; 1-否
coin	投币数	tinyint	N	
collect	是否收藏	tinyint	N	0-是; 1-否
play_time	最近播放时间	datetime	N	

续表 4.12 用户与视频关联表

字段名	字段解释	数据类型	允许空值	备注
love_time	最近点赞时间	datetime	Y	
coin_time	最近投币时间	datetime	Y	
collect_time	最近收藏时间	datetime	Y	

(13) 视频表 (video):

表 4.13 视频表

字段名	字段解释	数据类型	允许空值	备注
vid	ID 值	bigint unsigned	N	主键, 自增
uid	投稿用户 ID	bigint unsigned	N	
title	标题	varchar(80)	N	
type	类型	tinyint	N	0-自制; 1-转载
auth	作者声明	tinyint	N	0-无声明; 1-未 经允许禁止转 载
duration	播放总时长	double	N	
mc_id	主分区 ID	varchar(20)	N	
sc_id	子分区 ID	varchar(20)	N	0-是; 1-否
tags	标签	varchar(500)	Y	不同标签之间 用空格分隔
descr	简介	varchar(2000)	Y	
cover_url	封面 url	varchar(500)	N	
video_url	视频 url	varchar(500)	N	
status	状态	tinyint	N	0-审核中; 1-通 过审核; 2-未通 过审核; 3-已删 除
upload_date	上传时间	datetime	N	
delete_date	删除时间	datetime	Y	
visible	可见范围	tinyint	Y	0-公开; 1-仅自 己可见

(14) 视频数据表 (video_stats):

表 4.14 视频数据表

字段名	字段解释	数据类型	允许空值	备注
vid	ID 值	bigint unsigned	N	主键

续表 4.14 视频数据表

字段名	字段解释	数据类型	允许空值	备注
play	播放量	int	N	
danmu	弹幕数	int	N	
good	点赞数	int	N	
bad	点踩数	int	N	
coin	投币数	int	N	
collect	收藏数	int	N	
share	分享数	int	N	
comment	评论数量统计	int	Y	

4.3 用户画像引擎设计与实现

4.3.1 用户画像引擎设计

在本文 2.3 小节已介绍过用户画像的常见构建步骤,本文将参考该流程设计用户画像引擎。

用户画像引擎的数据来源主要为 MySQL,包括用户的部分基本信息、用户发送过的弹幕和与这些弹幕有关联的视频信息。这些信息主要产生自用户的信息行为,比如用户若发送了一条弹幕,前端会自动发送网络请求,通知后端业务服务将该弹幕录入数据库,为后续创建该用户的用户画像做准备。

为了降低开发和维护成本,本文拟将用户拥有的标签存储在 MySQL 数据库中,其物理表结构设计成如 4.2.2 小节中用户标签表 (user_tag) 所示。表中每一个元组代表某位用户拥有的某个维度上的标签,若干行用户 ID 相同的元组,即标签,构成该用户的用户画像。

接下来需要将这些用户数据以标签形式表达出来,便于算法计算和立体描述用户特征。本文的标签体系架构如下图所示:

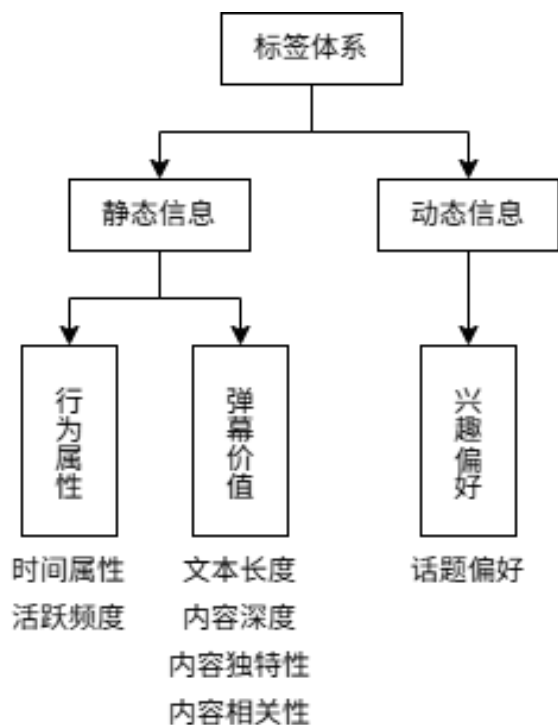


图 4.7 标签体系架构

引擎构建画像的核心目的是强化弹幕推送机制，所以标签体系将围绕弹幕信息构建量化评价机制，拟从行为属性、弹幕质量和兴趣偏好三大维度构建用户画像。

其中行为属性标签主要由用户发送弹幕的活跃频度和用户发送弹幕的活跃时段这两类信息转化而来。弹幕质量标签的信息来源包括弹幕文本长度、弹幕内容深度、弹幕内容独特性和弹幕文本与视频内容相关度。这两大维度的标签的主要信息来源均为相对不易更改的静态信息，例如弹幕元组的发布时间字段（create_date）、内容字段（content）等。

兴趣偏好标签通过解析用户的话题偏好信息获得，用户在一段时间内的话题关注点会动态变化，需要实时通过弹幕上下文信息推算。

明确了标签体系后，可以开始着手构建用户画像。系统在用户登录或通过刷新令牌（refresh token）刷新登录态后，向引擎发起构建用户画像的请求。总体业务流程如下：

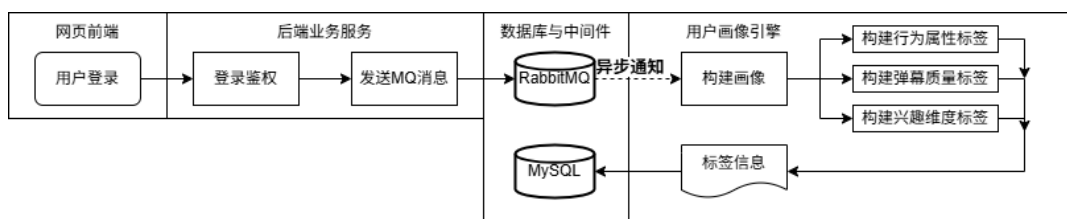


图 4.8 构建用户画像总体业务流程

一个用户的画像并不会短时间内高频变化，例如三个维度中变化频率较高的兴趣维度通常认为数周甚至数月才会发生变化，本文的登录认证采用 JWT 令牌机制，短期令牌的有效时间为 12 个小时，长期令牌的有效时间为 3 天，登录行为的触发频率足以覆盖用户画像的周期变化。

由于用户画像业务对时效性要求不高，且计算过程较为冗长，因此采用 RabbitMQ 异步传递通知，将引擎和上游流程解耦，缩短请求响应时间。

三个维度标签的构建过程相互独立，几乎没有耦合，因此为了提高计算速度，每个维度独占一条线程，实现并发运算，由任务末端数据收集器等待三条线程全部执行完后将标签结果统一持久化到数据库。在正式构建标签前，将采用 jieba 分词工具对文本数据进行分词处理，并基于预定义的停用词表过滤无意义词汇。如有需要，为进一步提取文本核心语义，还会利用 TF-IDF 算法计算词项权重，筛选出 Top-K 关键词用于后续标签匹配。

用户画像构建是其他与画像相关的业务的基础，其生成的标签数据将帮助其他业务实现应用落地。三个维度标签的算法流程解析如下：

(1) 构建行为属性标签：

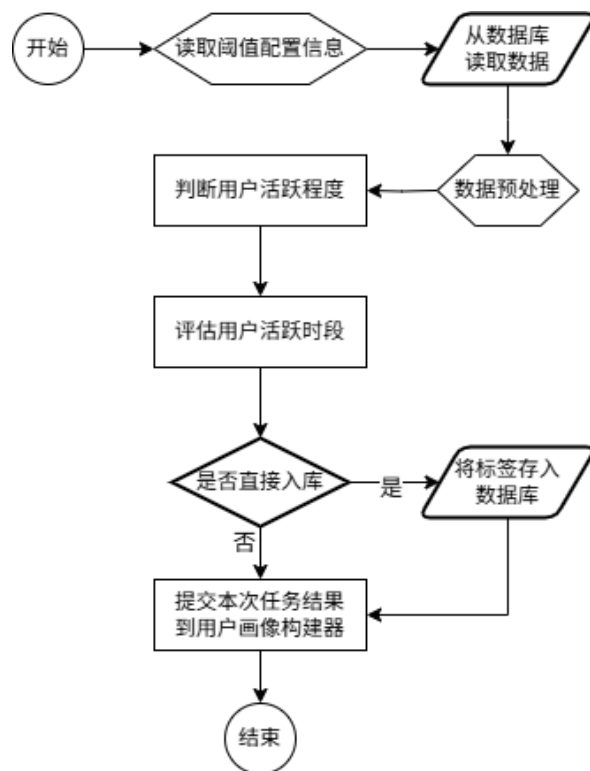


图 4.9 构建行为属性标签算法流程

引擎通过统计用户发送弹幕的总次数判断其是否活跃，当次数大于某个阈值，认定其活跃，小于某个阈值，认定其不活跃。评估活跃时段则是统计用户在某个时段发送的弹幕数量的占比，并比对阈值提取标签。可见算法执

行过程中需要读取阈值配置信息，这类信息需要提前计算好并放入数据库，并且阈值今后最好能根据整体数据变化动态调整。解决方案是准备两份数据：初始和默认数据采用本地 Json 文件存储，系统管理员必须要在早于标签任务启动的时间点提前准备好初始数据存入数据库，以防标签任务启动后读取到空值；还需要启动平行的定时任务，每隔一段时间自动根据总体数据更新阈值并将结果持久化到 Redis 数据库。读取阈值配置时，算法会先读取 Redis 数据库里较新的数据，如果读取失败，再读取本地 Json 文件的初始数据。

系统主要通过统计分析选取阈值，比如依据帕累托原则，20%的用户贡献 80%的行为量^[24]，因此高活跃用户应选在分布尾部，算法将发布弹幕的数量按用户 ID 分组，计算四分位数，当某个用户的总弹幕数大于等于 75 百分位数时认为其是“弹幕互动积极分子”。

(2) 构建弹幕质量标签：

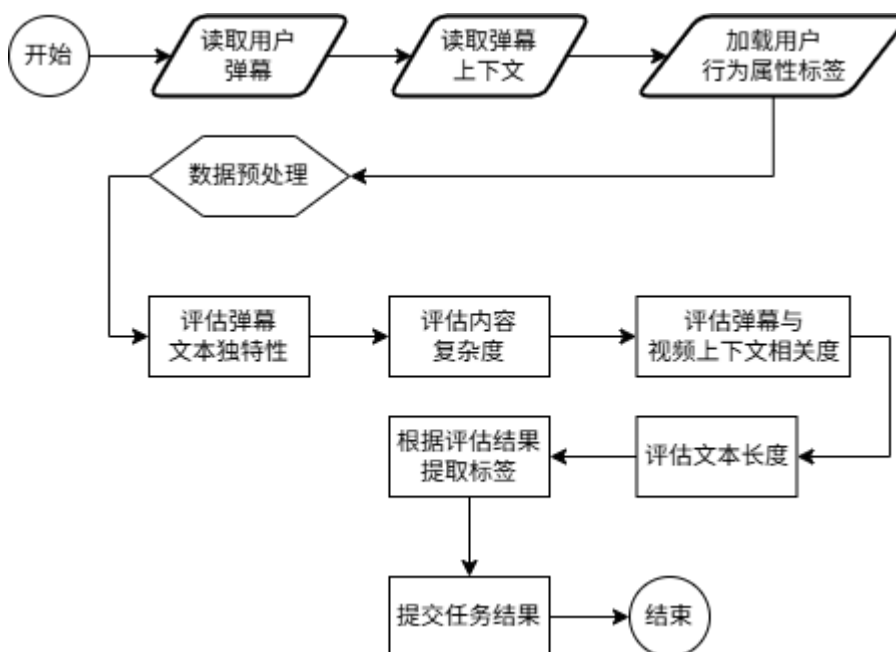


图 4.10 构建弹幕质量标签算法流程图

算法采用多维度加权计算每条弹幕的得分，其中文本长度基于词数判断，设置权重 0.2；专业度（内容复杂度）将分词后的弹幕文本与预先设定好的静态专业词库比对，匹配越多得分越高，该维度设置权重 0.3；弹幕相关度主要指弹幕文本与视频上下文（比如视频主题、标签和简介等）的相关性，采用关键词匹配计算，设置权重 0.2；文本独特性的一大目的是为了在推送时排除重复弹幕，设置权重 0.3，计算时先通过 TF-IDF 算法将所有弹幕转为向量矩阵，再拿出每条弹幕的向量矩阵与其他弹幕两两比对余弦相似度，每比对一次累加相似度得分。最终整体质量评分为各个属性得分的加权综合。

所有弹幕计算完毕后需要汇总得到平均得分，根据文本长度得分和平均得分，通过比对预先设定好的静态阈值提取弹幕质量标签。

(3) 构建用户兴趣标签：

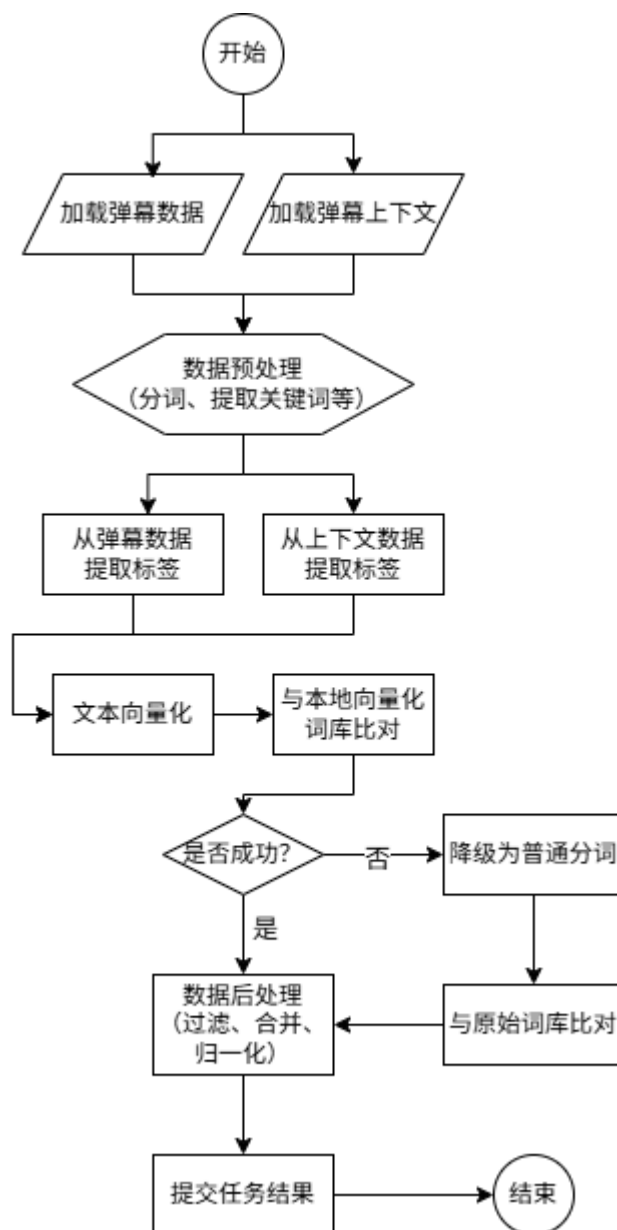


图 4.11 构建用户兴趣标签算法流程图

算法分别从弹幕文本和与弹幕相关的上下文信息（弹幕所在视频的标题、简介、标签等）提取标签，在提交结果前需要针对标签数据进行过滤、合并和归一化等处理。

算法会预先准备好两份与各类兴趣话题相关的词库，一份存储在本地 Json 文件作为备用，另一份经过向量化后存储在 Elasticsearch。获取到弹幕文本或者上下文信息后，将比较它们与各个话题的词库相似度，从中选出最为

相似的话题作为用户的兴趣标签。算法准备了两套计算机制：一套机制会先将数据上传到阿里云百炼平台部署的 text-embedding-v4 向量大模型，获取语义精度更高的向量数据，再通过余弦相似度算法与本地向量数据库匹配；另一套机制采用基础的分词匹配，当模型调用失败时，会自动切换到该方案。比对结束后即可获取到初步的兴趣标签。

为了反映时间对用户兴趣的影响，引擎基于 Ebbinghaus 遗忘曲线理论^[25]对初步筛选出来的标签进行处理，若标签来自距今时间较为久远的弹幕或上下文信息，将被赋予更低的权重。随后算法将过滤掉权重为 0 的冗余标签，并累加两类信息源的标签权重，实现标签合并。最后再将权重归一化，防止兴趣维度标签权重过高干扰其他维度标签的效力。

词库的向量化信息同样需要在标签提取流程启动前预先设定好，所以引擎管理员需要在本地 Json 文件准备好默认词库，并启动定时任务定期将动态变化的词库向量化到 Elasticsearch。

以上为构建用户画像的设计介绍。构建用户画像是引擎提供的基础服务，在本文的弹幕交互功能中，基于画像的应用主要体现为 4.1 小节介绍过的三点：查看与自己相似的用户、根据指定标签筛选弹幕和查看指定用户的弹幕。其中第一个功能由引擎实现，剩余两个功能由后端业务服务实现。

查看与自己相似的用户整体业务流程如下：

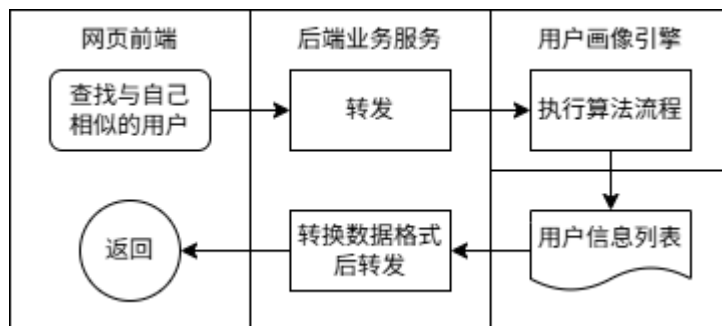


图 4.12 查看与自己相似的用户业务流程图

该功能将帮助用户查找同好。算法流程如下：

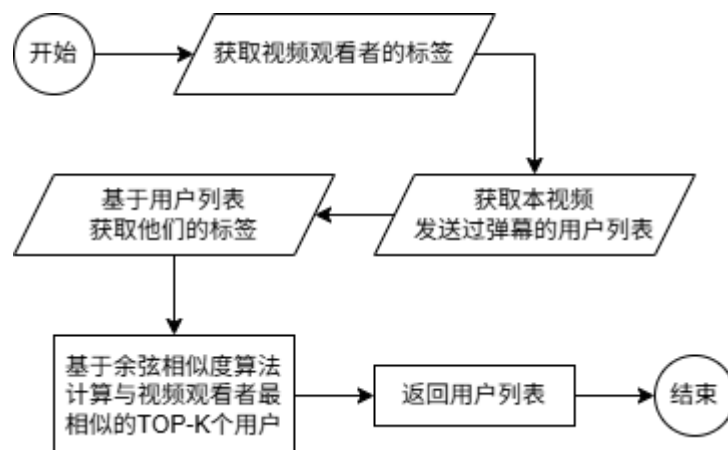


图 4.13 查看与自己相似的用户算法流程图

根据指定标签筛选弹幕和查看指定用户的弹幕这两个功能仅需根据参数利用 Mybatis-Plus 编写动态 SQL 语句从数据库获取数据即可完成。根据指定标签筛选弹幕可快速提取高质用户发布的弹幕，而查看指定用户的弹幕为查找相似用户功能的延伸，帮助用户快速锁定同好的观点。二者业务流程相似，可抽象为如下图所示的流程：

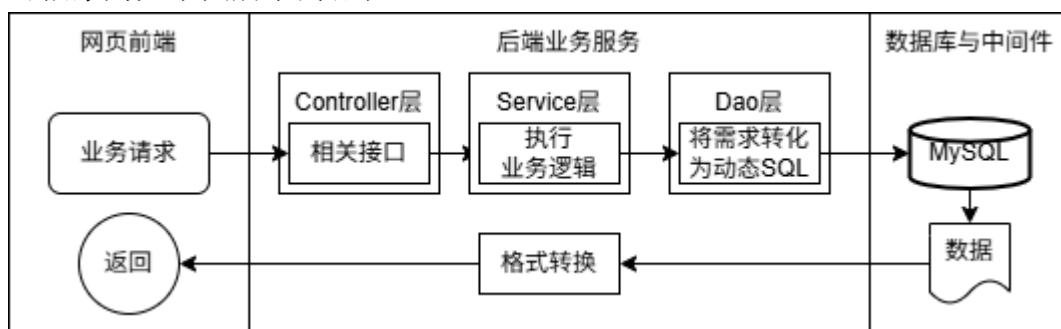


图 4.14 筛选弹幕通用业务流程图

4.3.2 用户画像引擎实现

引擎项目目录结构如下表所示：

📄	.env (61.0B)
📄	.gitignore (577.0B)
📁	Assets/
📄	behaviorAnalysis.json (1.1KB)
📁	docs/
📄	备忘录.md (10.5KB)
📄	professionalWords.json (401.0B)
📄	qualityThreshold.json (209.0B)
📄	recommend.json (122.0B)
📄	stopwords.json (3.6KB)

```
tags.json (3.1KB)
README.md (0.0B)
requirements.txt (134.0B)
src/
  __init__.py (127.0B)
  algorithm/
    __init__.py (127.0B)
    behaviorAnalyzer.py (6.9KB)
    behaviorTag.py (5.9KB)
    danmuScore.py (3.4KB)
    interestTag.py (9.1KB)
    qualityAnalyzer.py (947.0B)
    qualityTag.py (7.5KB)
    recommender.py (3.9KB)
    similarUser.py (4.2KB)
    vectorization.py (7.7KB)
  common/
    __init__.py (127.0B)
    qualityTagConstant.py (228.0B)
    redisConstants.py (218.0B)
  config/
    __init__.py (127.0B)
    application.py (2.8KB)
  main.py (158.0B)
  server/
    __init__.py (127.0B)
    app.py (4.2KB)
  util/
    __init__.py (127.0B)
    database.py (5.5KB)
    jsonHandler.py (872.0B)
    llmClient.py (5.4KB)
    profileBuilder.py (7.3KB)
    rabbitmq.py (5.6KB)
    scheduledJobs.py (1.4KB)
    spider.py (691.0B)
    wordHandler.py (4.5KB)
```

其中 src 目录为项目核心目录，存放算法的具体实现（algorithm 目录）、

工具代码（`util`）、关键配置信息（`config`）、公用静态信息（`common`）、Web 应用程序核心目录（`server`，包括 `app.py` 等入口文件）和应用程序启动文件（`main.py`）。Assets 作为系统的资源存储目录，集中管理算法运行所需的各类静态配置与词库文件。`requirements.txt` 用于记录 Python 项目所需依赖库及其版本。`README.md` 记录部署教程和项目介绍。`.env` 文件是系统的环境变量配置文件，遵循 Python-dotenv 规范，用于存储不应硬编码在源代码中的敏感信息，如 API 密钥、数据库密码等。`.gitignore` 文件是 Git 版本控制系统的忽略规则配置文件，用于指定不应被纳入版本控制的文件或目录。

引擎本身也作为独立 Web 服务，基于 Flask 框架构建，与后端业务服务的通信方式包括通过 RabbitMQ 中间件通信、通过调用 RESTful API 通信和通过共享 Redis 内存数据库通信三种。通过 RabbitMQ 中间件通信时，引擎主要作为消费者，其监听者代码示例如下：

```
def startListen(self):
    """开始监听队列"""
    if not self.connect():
        print("无法连接 RabbitMQ")
        return
    try:
        # 声明队列和交换机
        exchange_name = 'userProfile.exchange'
        self.channel.exchange_declare(exchange=exchange_name, exchange_type='direct',
durable=True)
        self.channel.queue_declare(queue='userProfile.queue1', durable=True)
        self.channel.queue_bind(exchange=exchange_name, queue='userProfile.queue1',
routing_key='userProfile')
        # 设置消费者
        self.channel.basic_consume(queue='userProfile.queue1',
on_message_callback=self.processorAsync, auto_ack=False)
        print(f"RabbitMQ 监听器已启动")
        self.channel.start_consuming()
    except Exception as e:
        print(f"监听过程出错: {e}")
    finally:
        self.stop()
```

引擎目前对外暴露一个可用接口，该接口用于查询与指定用户最相似的 Top-K 个用户，K 默认值为 5，算法采用余弦相似度比对用户标签。接口代码如下表所示：

```

@app.route('/findSimilar/<int:video_id>/<int:user_id>', methods=['GET'])
def findSimilar(video_id, user_id):
    """
    查找与当前用户相似的用户
    """
    try:
        uids = startSimilar(video_id, user_id, 5)
        return jsonify(uids)
    except Exception as e:
        print(f"查找相似用户失败: {e}")
        return jsonify([])

```

提取标签的核心算法代码较长，本文难以展示全流程代码，下表为提取用户兴趣标签的核心伪代码：

```

Algorithm: 用户兴趣标签生成算法
Input: 用户 ID uid, 时间衰减开关 use_time_decay, 归一化开关 normalize
Output: 兴趣标签集合

weights ← initializeWeights(SEMANTIC_TAGS) // 初始化权重字典
// 从观看视频中提取兴趣
user_videos ← queryUserVideos(uid) // 查询用户观看记录
video_texts ← [join(tags) for each video in user_videos]
batch_matches ← batchMatchText(video_texts) // 批量语义匹配
for each (video, matches) in zip(user_videos, batch_matches):
    interaction_weight ← calcInteractionWeight(video.play, video.love, video.coin,
    video.collect)
    time_factor ← calcTimeDecay(video.play_time) // 时间衰减
    for each (tag, match_count) in matches:
        weights[tag] += interaction_weight × match_count × 0.6 × time_factor
// 从弹幕中提取兴趣
danmakus ← queryUserDanmakus(uid) // 查询用户弹幕
danmaku_texts ← [d.content for d in danmakus]
batch_matches ← batchMatchText(danmaku_texts) // 批量语义匹配
now ← currentTime()
for each (danmaku, matches) in zip(danmakus, batch_matches):
    time_factor ← calcTimeDecay(danmaku.create_date, now)
    length_factor ← calcLengthWeight(len(danmaku.content))
    for each (tag, match_count) in matches:
        weights[tag] += match_count × time_factor × length_factor

```

```
// 过滤与归一化
result ← {tag: w for tag,w in weights if w > 0}
if normalize then
    max_w ← max(result.values())
    result ← {tag: round(w/max_w, 4) for tag,w in result}
end if
saveToDB(uid, result) // 保存到数据库
return result
```

用户画像引擎的可视化实现效果将在第 5 章展示。

4.4 后端业务服务的设计与实现

本文基于 Spring Cloud Alibaba 构建微服务系统，Spring Cloud Alibaba 内含 Nacos、Gateway 等依赖，可以帮助开发者快速构建网关服务和微服务配置中心。微服务模块划分如下图所示：

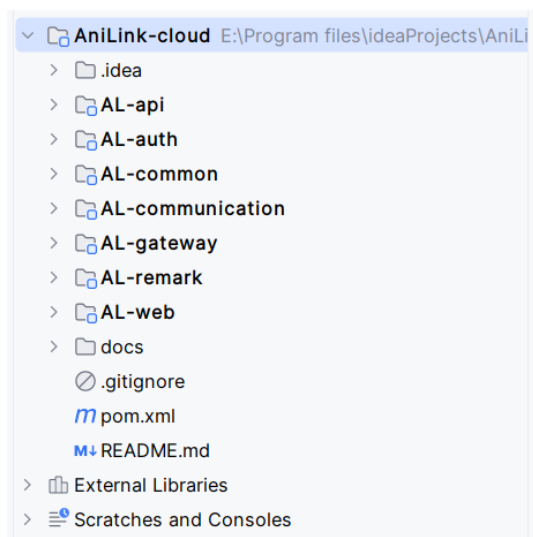


图 4.15 微服务模块划分示意图

每个模块采用 Spring Boot 3.0.5 构建 Web 应用程序。AL-web 模块集中了网站大部分业务实现，包括个人主页模块、Web 站点服务模块、视频播放页模块等。AL-remark 模块负责接受点赞、投币等互动请求，向其他具体业务模块分发任务。AL-gateway 模块为服务的网关层，负责负载均衡和全局权限鉴权等工作。AL-communication 模块为通信模块，利用 Netty 框架封装了基于 WebSocket 协议的通信业务，主要用于实现一对一聊天和全局消息广播等功能。AL-api、AL-auth 和 AL-common 模块为公共的工具模块，不作为独立的服务节点，通常作为业务模块的依赖引入。其中 AL-api 包含各种类型的公用实体类和枚举类，AL-auth 封装了公共拦截器，发挥作用的时机介于网关层和具体

业务模块之间，当网络请求成功通过网关层校验后，在流入各个业务模块前，会进入公共拦截器做更细粒度的校验。此外还支持对网络请求内含的信息进行修改。AL-common 内含工具方法、静态常量和通用配置信息等。

为了尽可能满足 3.2 小节提出的非功能性需求，后端业务服务部分功能做了额外的设计，接下来将介绍其中的代表性例子。

4.4.1 登录鉴权模块设计与实现

系统基于常用的 JWT 令牌技术进行登录鉴权，并且采用双 token 机制，设置一长一短两个 token，短 token（access-token）有效期为 12 小时，用于常规 API 访问，通常在请求头中携带；长 token（refresh-token）有效期 3 天，仅用于针对短 token 的校验和刷新，通常存储在 cookie 中。当短 token 未过期时，用户凭此可正常访问服务；当短 token 过期时，直接访问服务会被拦截，此时若长 token 未过期，则系统会刷新两个 token，并自动帮用户登录，重新赋予用户正常访问的权限；若长 token 也过期，则清除本地存储的用户信息，拦截用户请求，要求用户手动重新登录。

该机制相较于传统单 token 鉴权存在两点优势：

（1）延长用户登录时间。如果没有长 token，每隔 12 个小时用户就要手动重新登录。引入长 token 后，用户在 3 天内均无需进行该动作，并且只要用户登录间隔不超过 3 天，登录状态理论上可以无限期续约，提升了操作便捷度，优化用户体验。

（2）结合全局唯一 ID 可以进一步提升 JWT 令牌安全性，具体做法是在生成长 token 的同时给该 token 生成全局唯一 ID，称为“jti”，一份经过复杂加密后存入 JWT 令牌负载，另一份存入服务器本地内存，比如虚拟机内存或者 Redis 缓存。每次校验长 token 时，会从 token 负载取出 jti 与本地暂存的 jti 比对，只有完全一致才不会被拦截，否则认定为非法 token。该机制可有效应对重放攻击。

以下是双 token 机制的登录鉴权时序图：

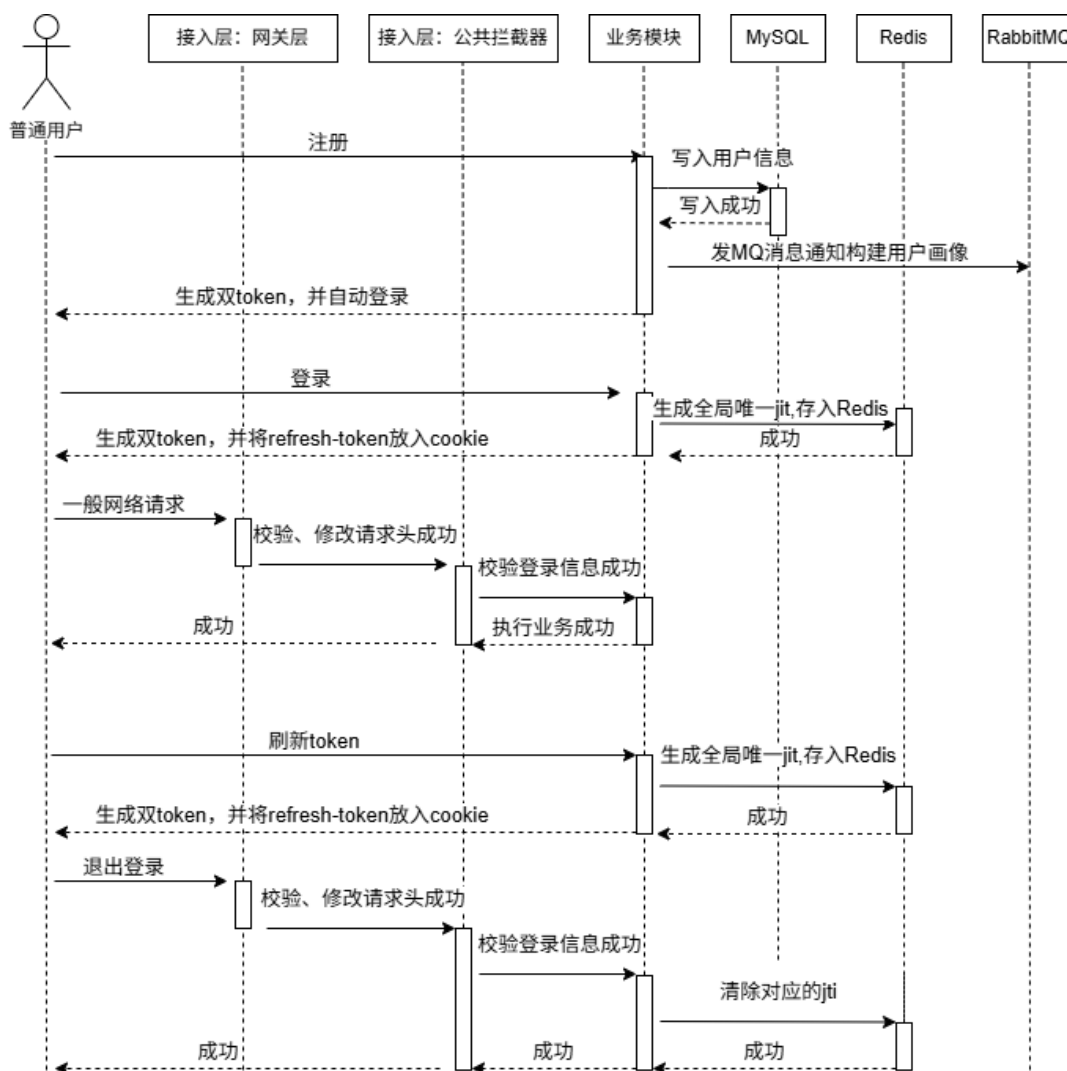


图 4.16 登录鉴权时序图

登录鉴权的网关层代码位于前文提到的 AL-gateway 模块，公共拦截器位于 AL-auth 模块的子模块 AL-auth-resource-sdk，通过 Spring 自动配置生效。主要登录鉴权接口和业务代码位于 AL-web 模块。由于登录鉴权代码较长，以下仅展示位于 AL-common 的 util 工具包的 JWT token 生成核心代码：

```

public static String createToken(Long userId, String username) {
    return Jwts.builder()
        .setSubject("USER_INFO")
        .setExpiration(new Date(System.currentTimeMillis() +
ACCESS_TOKEN_EXPIRATION))
        .claim(JWT_PAYLOAD_USER_ID, userId)
        .claim(JWT_PAYLOAD_USER_NAME, username)
        .signWith(tokenSignKey)
        .compact();
}

```

```

    }

    public static String createRefreshToken(Long userId, StringRedisTemplate template) {
        long jti = CodeUtil.geneVisibleCode(userId, System.currentTimeMillis());
        Duration ttl = Duration.ofMillis(REFRESH_TOKEN_EXPIRATION);
        String token = Jwts.builder()
            .setSubject("USER_INFO")
            .setExpiration(new Date(System.currentTimeMillis() + ttl.toMillis()))
            .claim(JWT_PAYLOAD_USER_ID, userId)
            .claim(JWT_PAYLOAD_JTI, jti)
            .signWith(tokenSignKey)
            .compact();

        // 缓存 jti, 有效期与 token 一致, 过期或删除 JTI 后, 对应的 refresh-token 失效
        template.opsForValue().set(JWT_REDIS_KEY_PREFIX + userId, Long.toString(jti), ttl);
        return token;
    }
}

```

4.4.2 点赞高并发优化设计与实现

常规的点赞实现是直接把点赞数据持久化到数据库，该实现的优点是开发简单，维护成本低，但是随着用户量的增长，点赞接口的并发量将越来越高，短时间内大量请求落到 MySQL 可能导致数据库宕机。为了解决该问题，本文将点赞业务流程设计如下：

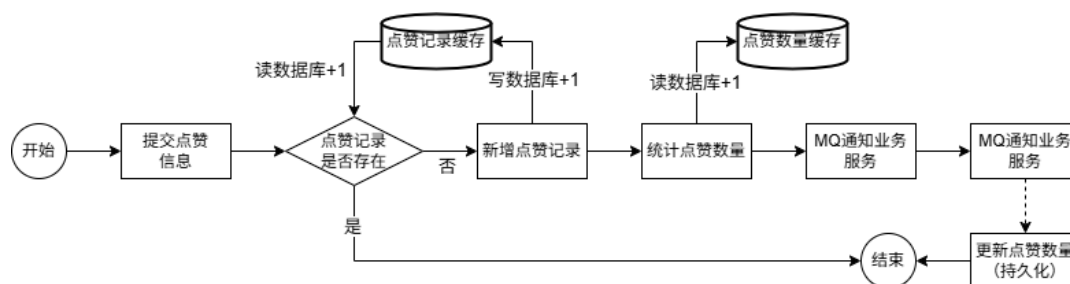


图 4.17 点赞业务流程图

AL-remark 模块负责流程中的缓存读写，读写完成后，通过发 MQ 消息通知具体的业务模块进行点赞数据持久化。如果该点赞是视频点赞，就通知视频业务，如果该点赞是评论/回复点赞，就通知评论/回复业务。

点赞记录缓存记录用户 ID 与视频 ID 的关联关系，以具体业务 ID（比如视频 ID、评论 ID）为键，采用 Redis 的 Set 数据结构，拥有自动去重的功能，可以将同一个用户对同一个视频的重复操作合并为一条操作，避免频繁发送 MQ 消息。

点赞数量缓存记录某个具体业务的点赞总数，其通过读取点赞记录缓存统计而来，选用 `SortedSet` 数据结构，使用业务类型标识作为外键，具体业务 ID 作为内键。`SortedSet` 同时具有哈希表和跳表的特征，可以根据某个值对数据进行排序，便于优先处理点赞数量较少、对点赞数变化敏感的业务。同时 `SortedSet` 通过 `ZPOPMIN` 命令可以实现查询和删除的原子化操作，当负责持久化的消费者集群部署时，为了避免数据重复消费，需要在查询点赞数后立即将数据从缓存中删去，但是高并发场景下查询和删除两条指令中间有可能被中断，而 `ZPOPMIN` 解决了这个问题。

系统采用 `Spring Task` 定时任务每隔 20 秒查询点赞数量缓存，如果有数据才将其封装为 MQ 消息发送给业务模块，并及时删除。点赞业务对数据一致性要求不高，上游业务并不需要关心数据持久化结果，因此利用 `RabbitMQ` 将持久化和上游业务解耦，压缩响应速度。`AL-remark` 模块实现点赞的核心代码如下：

```
@Override
public void doLike(LikeDTO likeDTO) {
    // 1.基于前端的参数,判断是执行点赞还是取消点赞
    boolean success = likeDTO.getLiked() ? like(likeDTO) : unlike(likeDTO);
    // 2.判断是否执行成功,如果失败,则直接结束。如果已经有了对应的用户 ID, 再次插入相同 ID 会失败, 从而避免重复刷点赞
    if (!success) {
        return;
    }
    // 3.如果执行成功,统计点赞总数
    Long likedTimes = redisTemplate.opsForSet()
        .size(LIKES_BIZ_KEY_PREFIX + likeDTO.getBizId());
    if (likedTimes == null || likedTimes == 0) {
        return;
    }
    // 4.缓存点总数到 Redis
    redisTemplate.opsForZSet().add(
        LIKES_TIMES_KEY_PREFIX + likeDTO.getBizType(),
        likeDTO.getBizId().toString(), likedTimes);
}

/**
 * 定时任务主函数
 */
```

```
@Override
public void readLikedTimesAndSendMessage(Byte bizType, int maxBizSize) {
    // 1.读取并移除 Redis 中缓存的点赞总数
    String key = LIKES_TIMES_KEY_PREFIX + bizType;
    // 优先处理点赞数小的
    Set<ZSetOperations.TypedTuple<String>> tuples =
redisTemplate.opsForZSet().popMin(key, maxBizSize);
    if (CollUtils.isEmpty(tuples)) {
        return;
    }
    // 2.数据转换
    List<LikedTimesDTO> list = new ArrayList<>(tuples.size());
    for (ZSetOperations.TypedTuple<String> tuple : tuples) {
        String bizId = tuple.getValue();
        Double likedTimes = tuple.getScore();
        if (bizId == null || likedTimes == null) {
            continue;
        }
        list.add(LikedTimesDTO.of(Long.valueOf(bizId), likedTimes.intValue()));
    }
    // 3.发送 MQ 消息
    mqHelper.send(LIKE_RECORD_EXCHANGE,
        StringUtils.format(LIKED_TIMES_KEY_TEMPLATE, bizType),
        list);
}
```

第5章 基于用户画像的弹幕视频网站测试

5.1 测试和开发环境概述

测试工具列表如下：

表 5.1 测试工具列表

工具名	版本号
Apache JMeter	5.6.3
Microsoft Edge 浏览器	146.0.3856.84(64 位)
Google Chrome 浏览器	146.0.7680.165(64 位)
Postman	12.4.2

开发硬件环境列表如下：

表 5.2 硬件环境列表

操作系统	处理器	内存	硬盘
Windows 11(25H2)	12th Gen Intel(R)	16GB	477GB
	Core(TM) i7-12700H		
	(2.30 GHz)		
CentOS Linux 7	14 核	16GB	40GB
	4 核		

开发软件环境如下：

表 5.3 软件环境列表

软件名	版本号
MySQL	8.0
Redis	6.2
RabbitMQ	4.2.3
Elasticsearch	8.12.2
Minio	RELEASE.2024-08-29T01-40-52Z
VMware Workstation 17 Pro	17.0.0 build-20800274
Docker	20.10.24

5.2 功能性测试

系统的功能性测试将借助黑盒测试方法，通过设计测试用例测试系统的

输入与输出，不关心内部实现细节^[26]。

5.2.1 用户画像功能性测试

下面将用可视化测试检测用户画像相关功能模块的完备性。可视化测试以浏览器模拟用户端环境，测试系统的输入输出，并检查可视化结果是否正确。测试操作为登录系统后模拟用户信息行为，进入视频页面点击各项功能按钮。预期结果为后端业务系统与用户画像引擎无故障、能正确执行业务逻辑、能正确处理输入输出；前端网页能正确显示结果。测试的部分结果如下图所示：

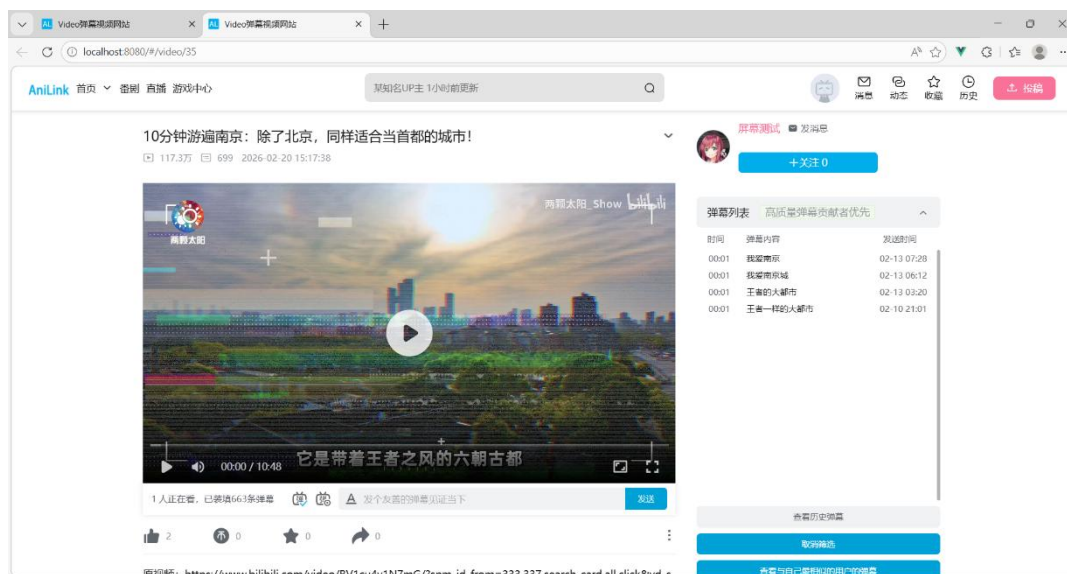


图 5.1 Edge 浏览器下用户画像模块可视化测试

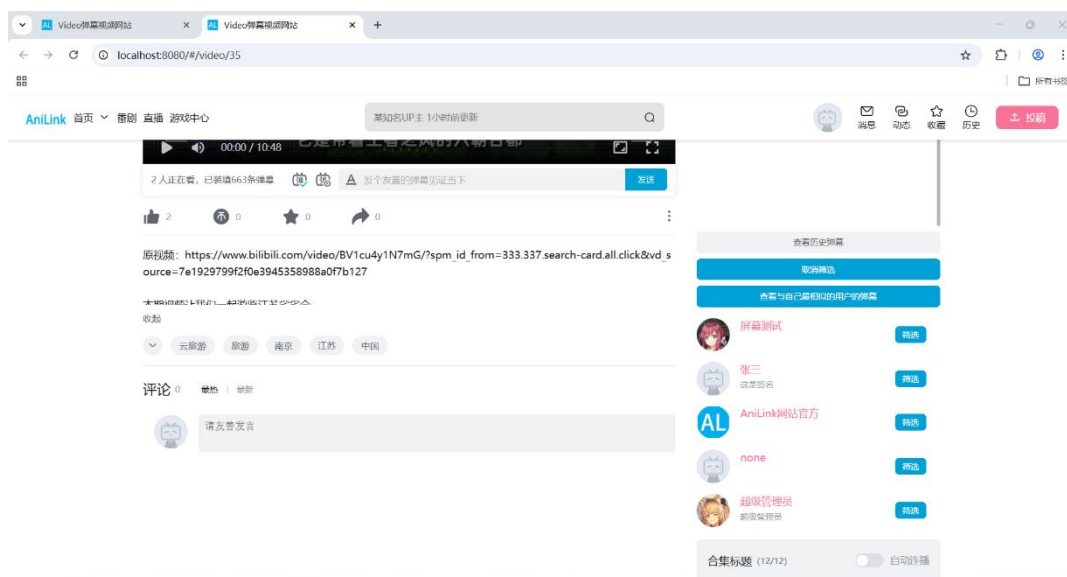


图 5.2 Chrome 浏览器下用户画像模块可视化测试

前端网页在两个浏览器均能正常显示测试结果，并且后端系统无故障。
下表为弹幕交互系统与用户画像相关的各功能测试结果一览：

表 5.4 弹幕交互模块中用户画像相关测试结果

功能/模块	正常情况	异常情况
查看与自己相似的用户	通过	失败
根据指定标签筛选弹幕	通过	失败
查看指定用户的弹幕	通过	失败

表中正常情况为输入正确参数时系统输出的情况，异常情况为故意输入错误或空参数时系统的输出情况，目的在于测试系统的鲁棒性。

5.2.2 后端业务功能性测试

登录鉴权模块的测试用例和测试结果如下表所示：

表 5.5 登录鉴权模块测试用例

测试项	操作步骤	预期结果	测试结果
正常登录	进入网站主页，在上方导航栏点击“登录”按钮，选择“登录”标签页，输入账户密码后点击下方“登录”按钮	登录成功	测试通过
注册	进入网站主页，在上方导航栏点击“登录”按钮，选择“注册”标签页，输入昵称和密码后点击下方“注册”按钮	注册成功，并自动登录成功	测试通过
登出	进入网站主页，鼠标悬停在头像，在弹出的弹窗点选“退出登录”	客户端清空缓存的用户信息，进入未登录状态，Redis 清空对应的 jti	测试通过
登录续约	等待 access-token 过期，再进入需要 token 才能访问的页面	请求被拦截，客户端重定向至主页，并自动重新登录	测试通过

续表 5.5 登录鉴权模块测试用例

测试项	操作步骤	预期结果	测试结果
拒绝续约	等待 access-token 和 refresh-token 均过期，再进入需要 token 才能访问的页面	请求被拦截，客户端重定向至主页，并解除登录状态	测试通过

表 5.6 登录鉴权模块测试结果

功能/模块	正常情况	异常情况
登录	通过	失败
注册	通过	失败
登出	通过	失败
登录续约	通过	失败

由于网站功能模块较多，接下来其它模块不再给出详细的测试用例。在类似的测试用例框架下，其它服务模块均能通过测试：

表 5.7 各模块测试结果

模块	正常情况	异常情况
Web 站点服务	通过	失败
个人主页	通过	失败
视频播放页	通过	失败
消息中心	通过	失败

5.3 非功能性测试

系统后端业务服务采用微服务架构，内部模块具有高内聚、低耦合的特点，新增业务模块对旧业务影响低甚至无影响，因此系统具有良好的扩展性。系统整体构成分布式系统，在接入层有负载均衡措施，每个服务节点容易扩展为集群部署，即便某个服务节点因故障宕机也能将网络请求分流到其它节点，保证系统整体平稳运行；系统所用中间件比如 Redis、RabbitMQ 等均配备完善的持久化机制，即便服务不慎宕机也能较快恢复数据；部分业务结合全局唯一 ID 和确认与重试机制（比如利用 RabbitMQ 的生产者/消费者确认与重试机制）避免数据重复消费，因此系统具有良好的可用性。

系统通过双 token 机制、MD5 加密传输等手段加强用户登录鉴权过程的数据安全。此外为了降低用户密码泄露风险，各业务在查询用户信息过程中

尽量不触及密码字段。因此系统具有良好的安全性。

接下来将使用 Apache JMeter 工具对系统部分高并发业务进行性能压测。在 Apache JMeter 中建立名为“点赞接口压测”的线程组，设置线程数 3000，Ramp-Up 时间为 1 秒，循环次数 1，其余参数设置如下表所示：

表 5.8 HTTP 请求（点赞）

参数名称	值
HTTP 请求名称	点赞
协议	http
服务器名称或 IP	localhost
端口号	8021
HTTP 请求类型	POST
路径	/remark/like
内容编码	UTF-8
跟随重定向	true
使用 KeepAlive	true
消息体数据	{ "bizId": 35, "bizType": 0, "liked": true}

表 5.9 HTTP 请求头管理器（点赞）

参数名称	值
Content-Type	application/json;charset=UTF-8
user-id	493181544250001

压测结果如下图所示：

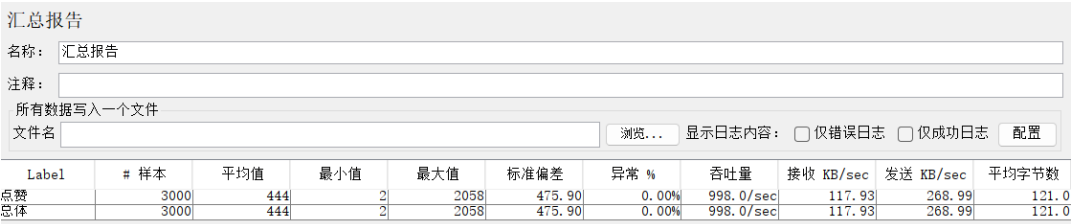


图 5.3 点赞接口压测

接口 QPS 达 998，异常值次数为 0，说明点赞业务面对瞬时高流量仍能保持高并发、高可用。

同理，针对进行了合并写优化的视频播放记录接口也进行压力测试：建立名为“记录视频播放接口压测”的线程组，设置线程数 1，Ramp-Up 时间为 1 秒，循环次数 100。其它参数：

表 5.10 HTTP 请求（提交视频播放进度）

参数名称	值
HTTP 请求名称	提交视频播放记录
协议	http
服务器名称或 IP	localhost
端口号	8022
HTTP 请求类型	POST
路径	/main/video/interact/record/add
内容编码	UTF-8
跟随重定向	true
使用 KeepAlive	true
消息体数据	{ "vid": 48, "duration": 138.0, "moment": "\${__Random(10,100,randomNumber)}, "commitTime": "2026-04-01 17:39:00"}

表 5.11 HTTP 请求头管理器（提交视频播放进度）

参数名称	值
Content-Type	application/json;charset=UTF-8
user-id	493181544250001

当线程组执行完毕时，理想中的情况是只有最后一条提交的记录被持久化，此前所有记录不予理睬。

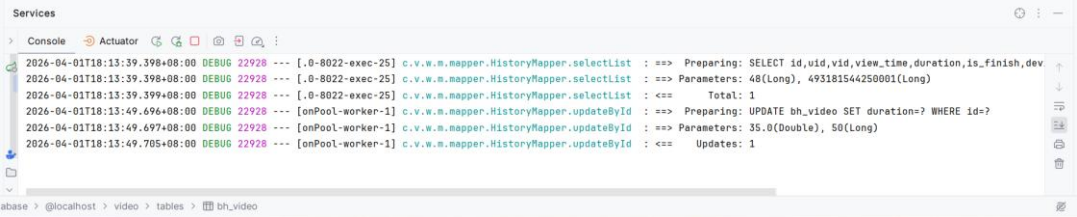


图 5.4 提交视频播放记录压测结果（a）

Label	# 样本	平均值	最小值	最大值	标准偏差	异常 %	吞吐量	接收 KB/sec	发送 KB/sec	平均字节数
提交视频播...	100	2	1	10	1.80	0.00%	335.6/sec	39.56	110.11	120.7
总体	100	2	1	10	1.80	0.00%	335.6/sec	39.56	110.11	120.7

图 5.4 提交视频播放记录压测结果（b）

结果显示，线程组执行完毕后，数据库只更新了一次，异常数为 0，符合

预期。说明提交点赞记录业务具有高可用特点。

第6章 总结与展望

本文梳理了弹幕视频网站的发展历史，分析了弹幕视频网站的市场需求和市场潜力，并在参考 MVC 设计思想、分层架构思想等的基础上，利用主流技术搭建了 B/S 架构的弹幕视频网站系统。在论文正文，则是详细分析了系统的设计、实现与测试。

此外通过文献查阅和亲历测评，发现市面上主流的弹幕视频网站的弹幕推送机制可能会让拥有特定信息需求的用户产生信息过载问题，因此本文设计并实现了基于用户画像的弹幕推送强化机制，旨在优化用户体验、更好满足用户特定的信息需求。

但是受限于本人的技术水平和有限的时间精力，本文所构建的系统仍有不少不足之处，因此在未来，现有系统可以做出如下改进：

（1）网站系统缺乏大文件加载优化机制，比如目前的视频页加载长视频是一次性从 Minio 服务下载完整的文件本体，容易导致下载失败或加载时间过长，影响用户体验。因此未来可引入分片传输机制，并借助 CDN 内容分发网络缓解源站压力。

（2）后端微服务模块划分存在不足。本文所构建的系统由单体架构重构而来，大部分旧业务仍集中在 AL-web 模块，服务拆分有待优化。未来可探索更高效的业务拆分。

（3）用户画像引擎缺乏完善的推荐效果评价机制，目前仅仅通过了功能测试，难以在更复杂的生产环境进一步验证其优化效果。未来可引入 A/B 测试机制，将用户随机分为两组：对照组使用传统推送，实验组使用基于用户画像的个性化推送。通过对比两组用户的弹幕系统使用率、平均观看时长和次日留存率，量化画像引擎的优化效果。此外，还需要能够获取真实用户数据的配套生产环境，解决方案之一是购买公网域名，将网站部署在可公开访问的云服务器。

（4）用户画像构建算法精度有待提高，例如构建用户标签时引入专门的大模型维护可动态扩展的词库，或者探索无词库的语义匹配方案。

参考文献

- [1] Cooper A, Robert Reimann R, Cronin D. About Face 3: The Essentials of Interaction Design[M]. New Jersey:Wiley Publishing Inc., 2007: 19-22.
- [2] 哔哩哔哩股份有限公司.Bilibili2025 中期报告[EB/OL].[2025-09-25].https://ea-cdn.eurolandir.com/press-releases-attachments./3924499/HKEX-EPS_20250925_11854920_0.PDF
- [3] 腾讯科技.盘点那些改变互联网的 20 大里程碑事件[EB/OL].[2013-01-17].<https://cio.zol.com.cn/349/3490857.html>
- [4] 百度百科“RealNetworks”[EB/OL].[2026-02-27].<https://baike.baidu.com/item/RealNetworks/1987003>
- [5] 姚延婷,王通.在线视频平台如何进行优质内容生态建设——以 B 站平台为例[J].传媒, 2024, (02): 70-72.
- [6] 杜善滨.PGC 率先发展下 UGC 盈利模式财务分析[D].上海: 东华大学, 2024.
- [7] 王佳琪.基于弹幕视频网站的弹幕文化研究[D].山东师范大学,2015.
- [8] 钟潇萌.基于个性化推荐的视频网站的设计与实现[D].武汉: 华中科技大学, 2022.
- [9] 姚苏梅.基于弹幕的在线视频社区群体用户画像模型研究[D].秦皇岛: 燕山大学,2021.
- [10] 郭强.基于 Bilibili 弹幕的“双碳”科普视频用户画像构建与分析[D].太原: 山西大学,2024.
- [11] 袁航.基于用户画像的商品推荐系统设计与实现[D].武汉: 华中科技大学, 2021.
- [12] SLOAN L, MORGAN J, BURNAP P, et al. Who Tweets? Deriving the Demographic Characteristics of Age, Occupation and Social Class from Twitter User Meta-Data[J]. PLOS ONE, 2015, 10(3): e0115545.
- [13] 张峰.应用 SpringBoot 改变 web 应用开发模式[J].科技创新与应用,2017,(23):193-194.
- [14] ESPOSITO C, CASTIGLIONE A, CHOO K K R. Challenges in Delivering Software in the Cloud as Microservices[J/OL]. IEEE Cloud Computing, 2016, 3(5): 10-14.
- [15] 张戥圣.基于 SpringCloud 的在线学习系统的设计与实现[D].武汉: 华中科技大学,2022.
- [16] 周立.Spring Cloud 与 Docker 微服务架构实战[M].电子工业出版社:201807:323.
- [17] 苟文博,于强.基于 MySQL 的数据管理系统设计与实现[J].电子设计工程,2017,25(06):62-65.
- [18] 袁浪情.基于 MVC 模式开发的高校食堂采购管理系统[J].中国质量,2023,(12):103-106.
- [19] 周贤武.基于 SpringBoot 的高并发网购平台系统及其后台管理的设计与实现[D].哈尔滨: 哈尔滨理工大学,2024.
- [20] 李若枫.基于用户画像技术的农产品新零售平台的设计与实现[D].重庆: 重庆三峡学院,2020.
- [21] 刘广东.基于“用户画像”的商品推送系统设计与实现[D].西安: 西安电子科技大学,2017.

- [22] Fowler M .企业应用架构模式[M].机械工业出版社:201005:688.
- [23] 轩兴涛.企业应用软件架构模式研究[J].农业网络信息,2007,(10):179-180+178.
- [24] 王玮.帕累托原则与阮冈纳赞图书馆学五定律[J].图书与情报,2002,(01):7-9.
- [25] 李桃迎,张鑫,陈燕.基于艾宾浩斯遗忘曲线的零售商品模糊关联分析[J].计算机应用研究,2018,35(02):462-465.
- [26] 高磊.基于用户画像的图书推荐系统设计与实现[D].石家庄: 河北经贸大学,2025.

致 谢

四年的大学本科生涯即将结束，回顾过往的学习经历，我很荣幸能够进入郑州大学信息管理学院，并选择分流到信息管理与信息系统方向。学院为本专业学子提供了充实且多样化的培养方案，扩展了学生的知识面，我因此得以接触到程序设计与信息管理系统的搭建，最终选择深入学习与软件工程相关的学识，并精进 Java 程序设计这一领域。

首先，我衷心感谢徐爱琴导师，徐老师平日里十分关心学生的个人成长，不论是课堂上还是课余时间，都尽力为学生的方向选择、课业学习和毕业论文提供指导。从论文开题到修改、定稿，徐老师也为我提供了许多宝贵的建议，让我得以提升写作能力，将毕业设计的每一个环节扎实推进到底。

我还要感谢学院里的诸位老师，他们恪尽职守，利用自己丰富的专业知识为学生传道授业解惑，帮助学生在各自的人生道路行稳致远。

最后，我要郑重感谢我的家人和好友，正是他们一直以来在精神和物质上的无私支持，我才得以趟过重重难关，实现一个又一个人生目标，并有底气在未知的将来乘风破浪。